

Opus Publica

RC1 Engineering Directive

The Constitution for Writing the Engineering Constitution

Version 1.0

Issued 09 August 2026

Document Type	Engineering Directive (Constitution-for-the-Constitution)
Issuing Authority	Office of the Chief Systems Architect, Opus Publica
Audience	Engineering Leadership Team — not developers, not build agents
Classification	Mandatory — Engineering Leadership
Effective Date	09 August 2026
Supersedes	None (first issuance)
Review Cadence	Quarterly, or upon RC milestone transition
Document ID	OP-DIR-RC1-001

This directive governs how the Engineering Constitution and Engineering Playbook SHALL be authored, certified, and maintained.

Preamble — Executive Order

This Directive is issued by the Office of the Chief Systems Architect of Opus Publica. It is not a specification of the platform. It is the specification of how the platform's specification SHALL be written. It is, in the precise sense intended by its authors, the Constitution for writing the Constitution.

Opus Publica has reached the point in its lifecycle where ad hoc engineering practice no longer scales. The platform has accumulated the familiar pathologies of a project built iteratively without a written contract between its leadership, its implementers, and its automated build agents: metadata mastered in more than one place, features that compile and pass static analysis yet fail at runtime, certifications asserted without objective evidence, and a recurring loop in which implementation outruns specification and is then corrected by another round of implementation. This Directive exists to terminate that loop permanently and to replace it with an enterprise engineering flow in which specification precedes implementation, and implementation is gated by runtime certification.

1. Authority

This Directive is issued under the authority of the Chief Systems Architect of Opus Publica and is binding on the entire engineering organization. It ranks immediately below the product vision and immediately above every other engineering document. Where any engineering document, code review comment, build-agent instruction, or operational runbook conflicts with this Directive, this Directive prevails until and unless the Directive itself is amended through the governance process defined in Chapter 4.16.

2. Audience

The intended audience of this Directive is the Engineering Leadership Team: the Chief Systems Architect, engineering managers, principal and staff engineers, the Head of Quality Assurance, and the Release Certification Authority. It is not written for individual contributors, and it is not written for the build agent ("Antigravity"). The Leadership Team is itself the audience because the Leadership Team is itself the author of the documents this Directive governs. The Engineering Constitution and Engineering Playbook produced under this Directive are the documents that will, in turn, govern implementers and the build agent.

3. Scope

This Directive governs the authoring, certification, and maintenance of every engineering document listed in Chapter 10 (Deliverables). It governs the engineering process by which any feature of Opus Publica is specified, implemented, verified, certified, and released. It governs the relationship between specification, implementation, runtime evidence, and production release. It does not govern product strategy, commercial policy, editorial policy, or scholarly content decisions, except where those decisions have an engineering consequence that must be reflected in the Engineering Constitution.

4. Effective Date and Application

This Directive takes effect on 09 August 2026. From that date forward, no implementation work on Opus Publica may proceed unless it satisfies the Production Acceptance Gate defined in this Preamble. Work already in flight at the effective date is granted a single transition window of thirty (30) calendar days, during which it must either be brought into compliance or be formally excepted in writing by the Chief Systems Architect. After the transition window, the Production Acceptance Gate applies without exception.

5. The Two Foundational Principles

Foundational Principle I — The Principle of Runtime Truth

No implementation SHALL be considered correct because it compiles, builds, or passes static analysis. Every user-visible feature MUST be verified through runtime execution and objective evidence. Where implementation, static analysis, and runtime behaviour disagree, runtime evidence SHALL take precedence until the discrepancy is resolved and the resolution recorded.

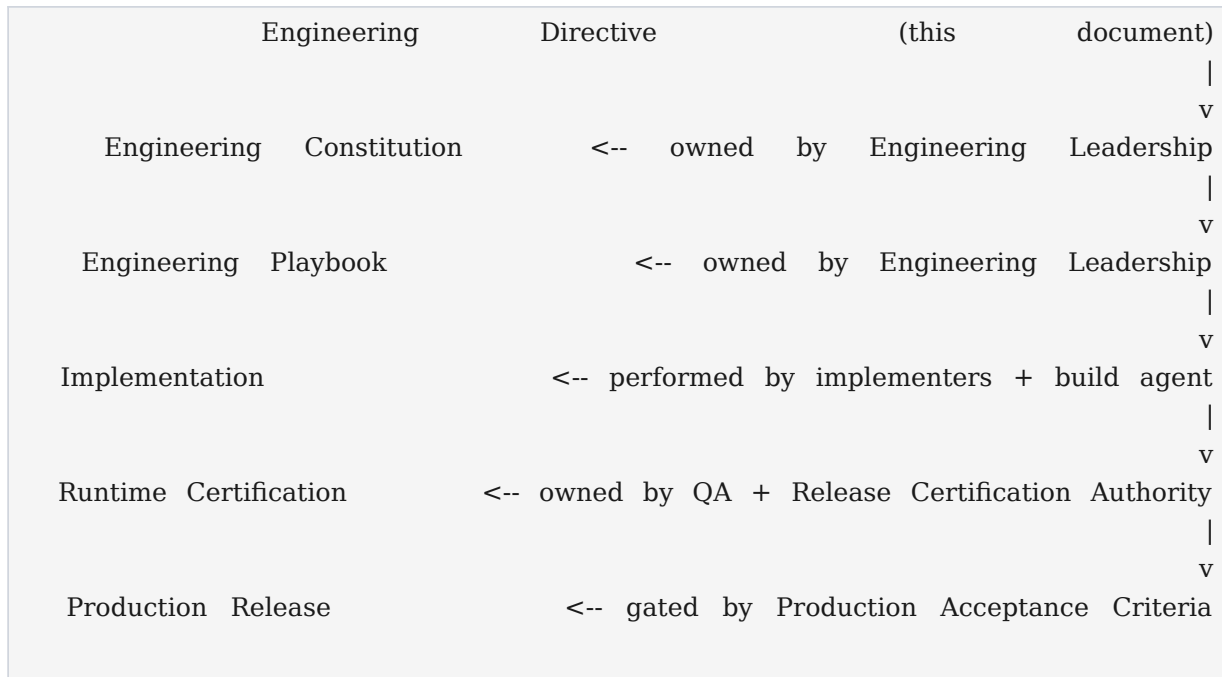
Foundational Principle II — The Production Acceptance Gate

No implementation MAY proceed to production release unless it satisfies ALL of the following, in this order: (1) the Engineering Constitution; (2) the Engineering Playbook; (3) Runtime Certification with objective evidence; (4) the Production Acceptance Criteria. Failure on any single criterion blocks release. There are no overrides below the Chief Systems Architect, and the Chief Systems Architect SHALL NOT override a Runtime Certification failure — the implementation SHALL be corrected instead.

These two principles are not stylistic preferences. They are the formal answer to the specific failure mode this project has exhibited: an implementation loop in which a build agent produces code, the code is observed to fail in production, the failure is patched, and the loop repeats without the underlying specification being corrected. Runtime Truth abolishes the implicit assumption that “it builds” is a sufficient condition of correctness. The Production Acceptance Gate abolishes the implicit assumption that implementation can proceed in the absence of a written, certified specification. Together they convert the engineering organization from a reactive bug-fixing service into a proactive specification-and-verification organization.

6. The New Engineering Flow

The Production Acceptance Gate replaces the previous informal flow with the following mandatory sequence. No step may be skipped. No step may be performed out of order. Each arrow represents a one-way dependency: a downstream step may not begin until the upstream step is signed off by its named owner.



The previous flow — in which a prompt to the build agent produced implementation, the implementation produced a bug, and the bug produced a fix — is formally retired. That flow is not permitted under this Directive. Any engineering activity that cannot be traced upward to a clause in the Engineering Constitution and Engineering Playbook is, by definition, out of process and SHALL be halted on discovery.

7. How to Read This Directive

This Directive uses the requirement vocabulary of IETF RFC 2119. The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” are used throughout with the precise meanings defined in RFC 2119. A compact reference is provided in Appendix A. Where this Directive uses “SHALL”, the Engineering Leadership Team is contractually bound to comply; where it uses “MUST”, the resulting Engineering Constitution SHALL itself use “MUST” when specifying the same requirement to implementers.

Chapters 1 through 3 establish the mission, the engineering philosophy, and the non-negotiable rules that bind every downstream document. Chapters 4 and 5 enumerate the chapters that the Engineering Constitution and Engineering Playbook, respectively, MUST contain. Chapters 6 through 8 specify how those chapters MUST be written, how code MUST be written, and how completeness MUST be certified. Chapter 9 constrains how the platform may evolve in RC2 and beyond. Chapter 10 enumerates the deliverables. The appendices provide reference material that is normative where so indicated.

TABLE OF CONTENTS

Right-click the table below and select "Update Field" to refresh page numbers.

Chapter 1 — Mission

1.1 The Product in One Sentence

Opus Publica is a production-grade scholarly publishing operating system: a single, deterministic platform that owns the entire lifecycle of a scholarly work — from submission, through peer review, editing, production, publication, persistent identification, indexing, preservation, and long-term archiving — and that does so with the rigor, auditability, and operational discipline expected of mission-critical infrastructure.

1.2 What Opus Publica Is Becoming

Opus Publica is in transition from an early-stage publication venue operated by Advocacy Unified Network (AUN) into a reusable engineering platform. Today the public surface of the product exposes eight named journals, a books catalogue, an author submission portal, a Diamond Open Access policy stack aligned with the Committee on Publication Ethics (COPE), and Crossref/Crossmark integration. That surface is, however, only the visible layer of a platform whose engineering substrate is incomplete: editorial boards are sparse across most journals, DOI resolution is inconsistent, the preservation story is absent, JATS XML output is not produced, and the OAI-PMH endpoint required by aggregators does not exist. The mission of RC1 is to close the gap between the product's public posture and its engineering reality, and to do so under a written engineering contract that prevents the gap from reopening.

What Opus Publica is becoming, therefore, is not merely a faster or prettier journal website. It is becoming the operating system on which AUN — and, in time, other publishers — run the business of scholarly publishing. An operating system, in the engineering sense, is defined not by its user interface but by its contracts: the contracts it offers to its users, the contracts it enforces on its operators, and the contracts it imposes on its own implementation. This Directive establishes the meta-contract under which all of those lower-level contracts SHALL be written.

1.3 What Opus Publica Is Not

Precision about what the platform is not is as important as precision about what it is. The following exclusions are normative: they constrain the scope of the Engineering Constitution and SHALL NOT be silently eroded by individual feature work.

- **Opus Publica is not** a content management system or a general-purpose website builder. It is a scholarly publishing platform; any feature whose primary purpose is general web publishing is out of scope and SHALL be implemented outside the platform.
- **Opus Publica is not** another Open Journal Systems (OJS). OJS, maintained by the Public Knowledge Project (PKP), is the incumbent and the reference point, but Opus Publica is a clean-room reimplement with a different architecture, a different extension model, and a different operational discipline. Where OJS concepts are preserved (roles, workflow stages, section model),

they SHALL be preserved deliberately and documented; where they are replaced, the replacement SHALL be justified.

- **Opus Publica is not** a plugin marketplace. RC1 and RC2 ship no third-party plugin surface. A typed extension contract is a RC3 concern (Chapter 9) and SHALL NOT be reverse-engineered from ad hoc hooks added during RC1 implementation.
- **Opus Publica is not** a research information system (CRIS) or a current research information system. It interoperates with such systems via ORCID, Crossref, and OpenAIRE, but it does not own grants, funding, or institutional affiliation data as master records.
- **Opus Publica is not** an experiment in agentic software development. The build agent (Antigravity) is an implementation tool operating under this Directive; it is not a co-author of the specification, and its outputs are subject to the same certification regime as any other implementation.

1.4 The Scholarly Publishing Operating System, Defined

For the purposes of this Directive and all documents derived from it, a “scholarly publishing operating system” is a platform that satisfies the following seven definitional properties. Each property is a mandatory capability of RC1; absence of any property disqualifies a build from RC1 certification, regardless of how many individual features appear to function.

#	Definitional Property	Meaning
P1	Canonical Article Representation	Every published article has exactly one canonical, versioned, content-addressed representation (JATS XML, ANSI/NISO Z39.96-2021) from which all derived renderings (HTML, PDF, citations) are produced deterministically.
P2	Persistent Identification	Every published article has a typed set of persistent identifiers, of which at least one is a DOI registered with a recognized registration agency (Crossref or DataCite).
P3	Deterministic Workflow	Every editorial state transition (submission → review → decision → production → publication → correction → retraction) is a named, audited, idempotent operation with defined preconditions, postconditions, and side effects.

P4	Immutable Publication	Once an article version is published, its rendered artifacts (PDF, HTML, JATS, supplements) are immutable and content-addressed by SHA-256. Corrections and retractions produce new versions with explicit version relationships.
P5	External Interoperability	The platform exposes first-class integration with Crossref (deposit and REST), ORCID (public and member APIs), OAI-PMH (oai_dc and OpenAIRE), SWORD v3, and at least one preservation service (CLOCKSS, LOCKSS, or Portico).
P6	Full Auditability	Every state transition on a published or publishable work is recorded in an immutable audit log with actor, timestamp, before-state hash, after-state hash, and reason, in conformance with NIST SP 800-92.
P7	Operational Measurability	The platform exposes service-level indicators (SLIs) and service-level objectives (SLOs) for every externally visible operation, with error budgets, in conformance with the Google SRE discipline.

1.5 Mission Statement

Mission Statement

Opus Publica's mission is to operate as the production-grade scholarly publishing operating system of Advocacy Unified Network and its successors — a single, deterministic, auditable platform that owns the lifecycle of every scholarly work it publishes, that interoperates with the global scholarly infrastructure on equal terms with incumbent publishers, and that is engineered to a written contract enforceable at runtime.

1.6 Strategic Position Relative to OJS

Open Journal Systems (OJS), maintained by the Public Knowledge Project at Simon Fraser University, is the de facto incumbent among open-source journal platforms and operates approximately ten thousand journals worldwide. OJS is the reference architecture against which Opus Publica is positioned, and the Engineering Constitution SHALL explicitly document, for each OJS concept that Opus Publica preserves, the rationale for preservation and the fidelity of the mapping. This is not optional nostalgia: a documented mapping is what permits an OJS journal to migrate to Opus Publica without loss of editorial memory, and what permits the Engineering Constitution to demonstrate that Opus Publica is a successor rather than a parallel invention.

The strategic divergence from OJS is architectural, not functional. OJS is a PHP/Smarty monolith whose plugin hook system couples extension authors to internal call sites, whose schema migrations across major versions have historically been painful, and whose separation between publishing-domain logic and presentation is weak. Opus Publica replaces this with a service-decomposed architecture, typed APIs, schema-as-code migrations, and an extension model that is deliberately deferred to RC3 so that it can be designed against a stable internal contract rather than reverse-engineered from a monolith. The Constitution SHALL make this divergence explicit and SHALL forbid the re-introduction of OJS-style coupling under any pressure of short-term feature delivery.

1.7 RC1 Success Criteria

RC1 (“Release Candidate 1”) is the first build of Opus Publica that is certified against the Engineering Constitution and Engineering Playbook produced under this Directive. RC1 is not feature-complete relative to the long-term vision; it is contract-complete relative to a defined minimum scope. The success criteria for RC1 are the following, all of which MUST be satisfied and runtime-certified before RC1 is declared:

1. All twenty-four Constitution chapters listed in Chapter 4 of this Directive are authored, reviewed, and signed off by Engineering Leadership.
2. All ten Playbook chapters listed in Chapter 5 of this Directive are authored, reviewed, and signed off by Engineering Leadership.
3. The seven definitional properties (P1–P7) are each demonstrably satisfied by runtime evidence in a production-equivalent environment.
4. At least one end-to-end publication — submission through preservation deposit — has been executed against the platform and certified.
5. All eight deliverables listed in Chapter 10 exist, are version-controlled, and are signed off by their named owners.
6. The Production Acceptance Gate has been exercised on at least one feature and has produced a complete certification artifact set.

1.8 Out of Scope for RC1

The following capabilities are explicitly out of scope for RC1 and SHALL NOT be implemented in RC1 even under schedule pressure. Each is a candidate for RC2 or RC3 and is constrained by the future-proofing rules in Chapter 9.

- Third-party plugin SDK and plugin marketplace (RC3).
- AI-assisted editorial features (RC3, governed by Chapter 9.3).
- Multi-tenant hosting of unrelated publishers (RC2, after tenancy model is specified in the Constitution).
- COUNTER 5.1.1 SUSHI automated harvesting (RC2; COUNTER event recording is in RC1 scope, SUSHI serving is not).
- Full JATS round-trip editing UI (RC1 produces JATS from a structured internal model; interactive JATS authoring is RC2).
- Mobile-native applications (out of scope indefinitely; the platform is responsive-web only).

Chapter 2 — Engineering Philosophy

This chapter is the highest-leverage chapter in the Directive. Philosophy is the set of commitments that determine every downstream decision; an engineering organization that has not written its philosophy will re-litigate it in every code review, every architecture decision record, and every incident postmortem. The twelve principles below are not a menu. They are the philosophy of Opus Publica engineering, and the Engineering Constitution SHALL operationalize each of them as concrete, testable requirements. Where two principles appear to conflict, the precedence rules in section 2.14 resolve the conflict; ambiguity is not permitted.

2.1 Principle 1 — Single Source of Truth

Each datum that the platform owns is mastered in exactly one service and exactly one store. All other references to that datum are derived, read-only, and explicitly labelled as such. The canonical source for an article's published state is the publication store; the website, the indexing endpoints, and the preservation deposits all read from it and never write to it. The canonical source for an author's identity is the identity service; the editorial workflow, the ORCID write-back, and the Crossref deposit all consume that identity and never store a competing copy. This principle exists because the alternative — metadata mastered in multiple subsystems, as is the OJS pathology — produces divergent values that are discovered only when a reader, an indexer, or an auditor notices the discrepancy, by which point the scholarly record has been corrupted.

Operationalizing this principle requires the Engineering Constitution to enumerate every mastered datum and name its owning service. Where a datum must be cached for performance, the cache SHALL be labelled with its source-of-truth reference, a time-to-live, and a reconciliation strategy. Where a datum is replicated for resilience, the replicas SHALL be declared followers, not peers, and writes SHALL be routed exclusively to the leader. The Constitution SHALL forbid the pattern, common in immature platforms, in which a denormalized convenience field is silently treated as authoritative after a release or two.

2.2 Principle 2 — No Duplicate Logic, No Duplicate Metadata

Closely related to the Single Source of Truth but distinct: the Single Source of Truth constrains where data lives, while this principle constrains where logic lives and where metadata is expressed. Any computation that is performed in more than one place — DOI validation, author-name normalization, citation rendering, license derivation — is, by definition, a duplicated logic bug waiting to happen, because the two implementations will diverge the moment one of them is updated and the other is not. The Constitution SHALL require that every piece of domain logic have exactly one owning module, that every cross-cutting concern be implemented once and exposed through a single interface, and that any apparent need to duplicate logic be treated as a design smell requiring an abstraction.

The same applies to metadata. The DOI of an article is expressed once in the canonical record and is referenced everywhere else by identifier, never re-typed. The license of an article is expressed once and is referenced everywhere else by license identifier. The Constitution SHALL require that every metadata field have a single canonical location and that every other location that needs the value holds a reference (a foreign key, an identifier, a hash) rather than a copy. Where a copy is unavoidable for technical reasons (for example, in a deposited JATS XML artifact), the copy SHALL be regenerated from the canonical source at deposit time and never hand-edited.

2.3 Principle 3 — Deterministic Workflow

Every editorial state transition is a named, idempotent operation with explicitly defined preconditions, postconditions, and side effects. Given the same input state and the same operation, the resulting state SHALL be identical. This is the publishing analogue of a finite-state machine: an article is, at any moment, in exactly one editorial state, and the set of legal transitions out of that state is finite, named, and exhaustively enumerated. The Constitution SHALL publish the state machine as a first-class artifact — a diagram, a transition table, and a machine-readable specification — and SHALL forbid any implementation path that reaches a published state without traversing a named transition.

Determinism has operational consequences that the Playbook SHALL make explicit. Idempotent transitions mean that a retry (whether initiated by a user double-clicking, a network timeout, or a build-agent re-execution) MUST NOT advance the article through two transitions. Conditional updates using ETags and If-Match headers (RFC 9110 §13) SHALL be used on every state-mutating API to detect concurrent modification. Where a transition has external side effects (a Crossref deposit, a CLOCKSS deposit, an ORCID write-back), the side effect SHALL be modelled as an outbox event with at-least-once delivery and an idempotent consumer, so that the transition itself remains deterministic even when the external system is slow or unavailable.

2.4 Principle 4 — Immutable Publication

Once an article version is published, its rendered artifacts are immutable. The PDF, the HTML, the JATS XML, and every supplement are content-addressed by SHA-256 and are never modified in place. Corrections are published as new versions with explicit version relationships (version-of, supersedes, is-superseded-by). Retractions are published as a retraction notice linked to the retracted article, and the retracted article's rendering is updated to carry a prominent retraction banner — but the underlying article artifact is not altered; only its presentation layer is.

This principle is the engineering embodiment of a norm that scholarship has held for centuries: the published record is fixed. A reader who cites a paper in 2026 MUST be able to retrieve the same artifact in 2036 and verify that it is byte-identical to what they cited. The Constitution SHALL require that every published artifact be retrievable by content hash indefinitely, that every version relationship be bidirectional and queryable, and that any change to a published artifact (including a typo fix in a PDF) be

implemented as a new version with a recorded reason, never as a silent overwrite. The Principle of Immutable Publication is non-negotiable because it is the foundation of citability.

2.5 Principle 5 — Evidence Before Assumption

Every architectural claim, every performance target, every reliability commitment, and every “it works” assertion **MUST** be supported by objective evidence. The Evidence-Based Software Engineering (EBSE) methodology formalizes this as a discipline: claims are hypotheses, measurements are experiments, and conclusions are warranted only when the evidence supports them. The Playbook **SHALL** require that every Architecture Decision Record cite the evidence that supports its recommendation, every SLO be backed by a measurement plan, and every “complete” certification be accompanied by the runtime artifacts that constitute its evidence.

This principle exists because the alternative — assumption as a substitute for evidence — is the single most common root cause of the implementation loops this Directive is written to abolish. “It should work because the type system says so” is an assumption. “It works because here is the runtime log showing the state transition” is evidence. The Playbook **SHALL** train every implementer, every reviewer, and every build agent prompt to demand evidence and to reject assumption, and **SHALL** make the absence of evidence a blocking review comment rather than a polite suggestion.

2.6 Principle 6 — Runtime Verification over Static Analysis

Static analysis — type checking, linting, schema validation, build success — catches a class of errors at build time and is necessary, but it is not sufficient. Static analysis cannot verify that a Crossref deposit actually reaches Crossref, that a double-blind review file actually has author-identifying metadata stripped, that a published PDF actually renders correctly, or that an OAI-PMH harvest actually returns the expected records. These are runtime properties, and they **SHALL** be verified at runtime, continuously, against formal specifications. The discipline of runtime verification observes production traces against a formal specification and raises an alarm when the specification is violated; the Engineering Constitution **SHALL** identify the platform's critical invariants (for example: “every published article has exactly one DOI”, “every reviewer assignment has a conflict-of-interest declaration”) and **SHALL** require that each be monitored at runtime, not merely unit-tested.

Where implementation, static analysis, and runtime behaviour disagree, runtime evidence **SHALL** take precedence until the discrepancy is resolved and the resolution recorded. This is the operative content of Foundational Principle I (The Principle of Runtime Truth). It means, in practice, that a passing build is not a defence against a runtime failure: the runtime failure is the truth, the build is a hypothesis, and the hypothesis is falsified. The Playbook **SHALL** require that every runtime failure produce a recorded discrepancy between the hypothesis and the evidence, and that the discrepancy be resolved by changing either the implementation or the specification — never by suppressing the evidence.

2.7 Principle 7 — No Hidden State

All platform state is explicit, named, and observable. There is no implicit state carried in environment variables that are not declared, in database columns that are not documented, in feature flags that are not registered, in cached values whose source is not labelled, or in build-agent prompts that are not version-controlled. Hidden state is the medium through which the implementation loop propagates: a feature works on one developer's machine because of an undeclared environment variable, fails in CI because the variable is absent, and fails again in production for a third reason — and the absence of an explicit state model makes the discrepancy invisible until a user reports it.

The Constitution SHALL require a complete, version-controlled declaration of all platform state: configuration (per the 12-Factor App Factor III), feature flags (in a registered flag store with owners and lifecycles), cached values (with source, TTL, and invalidation strategy), and environmental dependencies (per Factor IV). The Playbook SHALL forbid the introduction of any state that is not declared, and SHALL require that every build-agent prompt that influences implementation be itself version-controlled and reviewable, because an unreviewed prompt is hidden state in the most literal sense.

2.8 Principle 8 — Idempotent APIs

Every state-mutating API exposed by the platform, and every external API the platform calls, SHALL be safe to retry. A network timeout, a user double-click, or a build-agent re-execution MUST NOT produce a duplicate Crossref deposit, a duplicate CLOCKSS deposit, a duplicate ORCID write-back, or a duplicate article version. Idempotency SHALL be achieved through the appropriate mechanism for each operation: conditional requests with ETags and If-None-Match / If-Match (RFC 9110 §13) for state-mutating platform APIs, and idempotency keys (in the style of payment processors) for outbound deposits to external services that do not natively support conditional requests.

The Constitution SHALL enumerate every state-mutating API and every outbound external call, and for each SHALL specify the idempotency mechanism, the key space, the retention period for idempotency records, and the conflict-resolution behaviour when a retry arrives with the same key but a different payload (the canonical answer is: reject with 409 Conflict and a pointer to the original response). The Playbook SHALL make idempotency a mandatory review checklist item for every new API, and SHALL forbid the merge of any API whose idempotency contract is not specified and tested.

2.9 Principle 9 — Everything Auditable

Every state transition on a published or publishable work — and every administrative action that affects identity, authorization, configuration, or external integration — SHALL be recorded in an immutable audit log. Each audit record SHALL contain, at minimum: the actor (authenticated principal or system service), the timestamp (UTC, monotonic within the platform), the operation performed, the before-state hash, the after-state hash, the reason (free text, but required), and a correlation id linking the transition to its triggering request. The audit log SHALL be append-only, SHALL be cryptographically

chained (each record referencing the hash of its predecessor) so that tampering is detectable, and SHALL be retained for the lifetime of the platform.

This principle operationalizes NIST SP 800-92 (Guide to Computer Security Log Management) as the platform's audit contract. The Constitution SHALL specify the audit log's schema, its storage, its retention, its access controls (read-only to all principals except a named audit officer), and its export interface for external auditors. The Playbook SHALL forbid any state transition that is not audited, and SHALL require that the audit log be verified for completeness as part of every release certification — a release whose audit log is incomplete is, by definition, uncertifiable.

2.10 Principle 10 — You Build It, You Run It

The engineer who writes a service retains operational ownership of it. The team that ships a feature carries the pager for it. This principle, articulated by Werner Vogels as the operational philosophy of Amazon Web Services, is the single most effective mechanism for aligning the incentives of implementation with the incentives of operation: an engineer who will be paged at 03:00 because of a feature they shipped writes more defensive code, better telemetry, cleaner error handling, and more honest documentation than an engineer who hands the feature to a separate operations team and walks away. For a publishing platform — where a deploy bug can corrupt the scholarly record — this alignment is not a productivity hack; it is a correctness requirement.

The Playbook SHALL assign every service and every feature a named owning team and a named on-call rotation, SHALL forbid the merger of any service without a declared owner, and SHALL require that every incident postmortem identify whether the owning team was on-call and, if not, why not. The Constitution SHALL specify the on-call handoff protocol, the escalation path, and the standing expectation that on-call engineers have authority to roll back any release they did not ship — a release that the on-call engineer cannot safely roll back is, by definition, not yet releasable.

2.11 Principle 11 — Conway's Law and the Inverse Conway Maneuver

Conway's Law states that organizations which design systems are constrained to produce designs that are copies of their communication structures. The law is empirical, not normative: it describes what happens, not what should happen. The Inverse Conway Maneuver is the normative response — design the team boundaries to produce the desired architecture, and the architecture will follow. Opus Publica's intended architecture is a set of services with clean, typed interfaces (submission, review, production, publication, indexing, preservation); the team structure SHALL mirror that service decomposition, or the architecture WILL erode under cross-team coordination pressure within a few release cycles. This is a law, not a preference, and the Constitution SHALL treat it as such.

Operationally, this means the Engineering Leadership Team SHALL assign each Constitution chapter (Chapter 4 of this Directive) to a named owning team, and that team's boundaries SHALL be the service boundaries. Where a single team owns multiple chapters, the team SHALL demonstrate that the chapters correspond to a single coherent service; where ownership is ambiguous, the ambiguity SHALL be

resolved before implementation begins, not discovered during integration. The Playbook SHALL forbid the assignment of a feature to a team that does not own the relevant Constitution chapter.

2.12 Principle 12 — 12-Factor Compliance

The platform complies with the Twelve-Factor App methodology as the baseline contract for a production-grade cloud application. Each of the twelve factors is a concrete engineering requirement, and the Constitution SHALL operationalize each as a testable rule. The factors of particular importance to Opus Publica are Factor III (configuration in the environment, not in the code — critical for per-tenant secrets such as Crossref credentials, ORCID OAuth client secrets, and CLOCKSS deposit credentials), Factor V (strict separation of build, release, and run — the foundation of immutable publication, because the release artifact is the content-addressed binary that is later certified), and Factor XI (logs as event streams — the foundation of the audit log and of observability).

The Constitution SHALL publish a 12-Factor compliance matrix that maps each factor to the specific platform mechanism that satisfies it, and the Playbook SHALL treat any deviation from 12-Factor as an architectural decision requiring a recorded justification. The default is compliance; deviation is the exception that requires evidence.

2.13 Principle Interactions and Conflict

The twelve principles are designed to be mutually reinforcing, but in practice they occasionally appear to conflict. The most common apparent conflict is between Principle 4 (Immutable Publication) and the practical need to fix a rendering bug in a published PDF. The resolution is that the immutability applies to the published artifact, not to the rendering pipeline: a rendering bug fix produces a new version of the artifact, with a recorded reason, and the new version is itself immutable. The Constitution SHALL specify the resolution for each anticipated conflict and SHALL require that any novel conflict be escalated to Engineering Leadership for resolution and recorded in the Constitution as a precedent.

2.14 Principle Precedence

When two principles genuinely conflict and the conflict is not pre-resolved in the Constitution, the following precedence applies, from highest to lowest:

7. Principle 6 (Runtime Verification over Static Analysis) — because runtime evidence is truth.
8. Principle 4 (Immutable Publication) — because the scholarly record is the platform's reason to exist.
9. Principle 9 (Everything Auditable) — because an unaudited transition is, for certification purposes, a transition that did not happen.
10. Principle 1 (Single Source of Truth) and Principle 2 (No Duplicate Logic) — jointly, because they are two faces of the same constraint.
11. Principle 8 (Idempotent APIs) — because retries are inevitable and must be safe.
12. Principle 3 (Deterministic Workflow) — because determinism is the composability of the platform.
13. Principle 7 (No Hidden State) — because hidden state is the medium of the implementation loop.

14. Principle 5 (Evidence Before Assumption) — because it is the meta-principle that activates the others.
15. Principle 10 (You Build It, You Run It) and Principle 11 (Conway's Law) — jointly, because they are organizational rather than technical.
16. Principle 12 (12-Factor Compliance) — the baseline, overridable only by explicit architectural decision.

This precedence is itself subject to the governance process and MAY be amended by the Chief Systems Architect. It is published here so that conflicts are resolved by reference rather than by argument.

Chapter 3 — Non-Negotiable Rules

3.1 Purpose of the Hard Contract

This chapter is the hard contract between the Chief Systems Architect and every implementer, every reviewer, every build agent, and every operator of Opus Publica. The rules below are not principles to be interpreted; they are prohibitions and obligations that admit no exception below the Chief Systems Architect, and several admit no exception at all. The Engineering Constitution SHALL reproduce each of these rules verbatim in the chapter that operationalizes it, and the Engineering Playbook SHALL provide the review checklist item that detects a violation. A build that violates any rule in this chapter is, by definition, not certifiable and SHALL NOT be released.

3.2 How These Rules Are Enforced

Enforcement is layered. At the lowest layer, automated checks (linters, schema validators, CI gates, runtime invariant monitors) SHALL detect violations of any rule that is machine-checkable. At the middle layer, code review SHALL apply the Playbook's review checklist, which enumerates each rule and the question that detects its violation. At the highest layer, Release Certification SHALL refuse to sign off any release in which a rule violation is known to exist. The Chief Systems Architect is the final escalation point for any proposed exception, and exceptions — where permitted — SHALL be recorded in a public exceptions register with a sunset date.

3.3 The Rules

The rules are organized into six groups: Architecture, Data, Publication, External Integration, Operations, and Process. Each rule carries a stable identifier (RULE-xxx) so that it can be referenced precisely in code review, certification, and incident postmortems.

3.3.1 Architecture Rules

RULE ARCH-001 No service MAY read or write a datum that is mastered by another service except through that service's published API. Direct cross-service database access is forbidden.

RULE ARCH-002 No service MAY be deployed without a declared owning team, a declared on-call rotation, and a declared SLO. Ownerless services are forbidden.

RULE ARCH-003 No API MAY be merged without an idempotency contract. The contract SHALL specify the mechanism (ETag, idempotency key, or naturally idempotent operation) and the test that verifies it.

RULE ARCH-004 No feature flag MAY be introduced without registration in the flag store, a declared owner, a declared lifecycle, and a sunset date. Undeclared flags are forbidden.

RULE ARCH-005 No build-agent prompt that influences implementation MAY be unversioned. Every prompt SHALL be stored in the same repository as the code it produces and reviewed as part of the pull request.

RULE ARCH-006 No synchronous call MAY cross a service boundary for an operation whose latency is not bounded by an SLO. Unbounded synchronous cross-service calls are forbidden.

RULE ARCH-007 No service MAY be implemented in more than one programming language for the same concern. Polyglot services are permitted only across distinct concerns (for example, a TypeScript front-end and a Python batch job), and the boundary SHALL be a typed interface.

3.3.2 Data Rules

RULE DATA-001 No datum MAY be mastered in more than one location. Every datum SHALL have a single canonical store; all other references SHALL be derived, read-only, and labelled with their source-of-truth reference.

RULE DATA-002 No schema change MAY be deployed without a forward-compatible migration plan and a rollback plan. Breaking migrations are forbidden without a documented deprecation window.

RULE DATA-003 No hand-written SQL MAY be merged where a typed query builder or ORM mapping exists for the same operation. Raw SQL is permitted only for operations the abstraction cannot express, and the reason SHALL be recorded in a comment.

RULE DATA-004 No production database MAY be accessed for read or write except through the service that owns it. Ad hoc database access for debugging SHALL use a read-replica and SHALL be time-boxed and audited.

RULE DATA-005 No mutable published artifact MAY exist. Every published artifact (PDF, HTML, JATS, supplement) SHALL be content-addressed by SHA-256 and immutable once written.

RULE DATA-006 No datum MAY be deleted except through a named, audited soft-delete transition. Hard deletes of mastered data are forbidden; purging is a separate, governed retention action.

RULE DATA-007 No migration MAY be implemented as a one-off script outside the migration framework. All schema and data migrations SHALL be version-controlled, idempotent, and reversible where reversibility is technically possible.

3.3.3 Publication Rules

RULE PUB-001 No article MAY reach the “published” state without a registered DOI. A published article without a DOI is, by definition, not published.

RULE PUB-002 No article MAY be published without a canonical JATS XML representation validated against the official NLM schema. Invalid JATS blocks publication.

RULE PUB-003 No published artifact MAY be modified in place. Corrections and retractions SHALL produce new versions with explicit version relationships; the original artifact SHALL remain retrievable by content hash indefinitely.

RULE PUB-004 No peer-review assignment MAY be created without a recorded conflict-of-interest declaration from the reviewer. Anonymous assignments without a Col declaration are forbidden.

RULE PUB-005 No reviewer-visible file in a double-blind review MAY contain author-identifying metadata. The platform SHALL strip such metadata at the platform layer, not rely on editorial convention.

RULE PUB-006 No retraction MAY be issued without a retraction notice that is itself a

published artifact, linked bidirectionally to the retracted article, and deposited to Crossref with the appropriate metadata.

RULE PUB-007 No version relationship MAY be implicit. Every version-of, supersedes, or is-superseded-by relationship SHALL be explicit, recorded, and queryable.

RULE PUB-008 No publication event MAY occur without a preservation deposit being queued. The deposit MAY be asynchronous, but the publication event and the preservation outbox event SHALL be committed atomically.

3.3.4 External Integration Rules

RULE EXT-001 No outbound call to Crossref, ORCID, CLOCKSS, LOCKSS, Portico, or any other external service MAY be non-idempotent. Every outbound deposit SHALL carry an idempotency key and SHALL be safe to retry.

RULE EXT-002 No external credential MAY be stored in code, in configuration files committed to version control, or in any non-secret store. All external credentials SHALL be held in the platform's secret manager and injected at runtime.

RULE EXT-003 No external service call MAY be made without a timeout, a retry policy, a circuit breaker, and an SLO. Unbounded external calls are forbidden.

RULE EXT-004 No DOI MAY be reused or reassigned. Once a DOI is minted for an article version, it is permanently bound to that version; corrections and retractions mint new DOIs for new versions.

RULE EXT-005 No ORCID iD MAY be auto-populated without user consent. ORCID OAuth SHALL be a three-legged flow initiated by the user; server-side population of ORCID data without consent is forbidden.

RULE EXT-006 No OAI-PMH endpoint MAY be unversioned. The endpoint SHALL support

incremental harvest via timestamps, SHALL implement resumption tokens correctly, and SHALL expose at minimum oai_dc and OpenAIRE-compliant metadata formats.

RULE EXT-007 No external integration MAY be considered complete without a recorded deposit receipt, a reconciliation job, and an alert on reconciliation failure.

3.3.5 Operations Rules

RULE OPS-001 No state transition MAY occur without an audit record. The audit record SHALL contain actor, timestamp, operation, before-state hash, after-state hash, reason, and correlation id.

RULE OPS-002 No audit log MAY be mutated or deleted. The audit log SHALL be append-only and cryptographically chained; tampering SHALL be detectable.

RULE OPS-003 No release MAY be deployed without Runtime Certification. A release without certification artifacts is, by definition, a non-release.

RULE OPS-004 No release MAY be deployed without a tested rollback path. A release whose rollback has not been exercised in a production-equivalent environment is forbidden.

RULE OPS-005 No production change MAY be made outside the deployment pipeline. Direct production mutation — by an engineer, by a build agent, or by a database console — is forbidden except through a named, audited break-glass procedure.

RULE OPS-006 No service MAY run without telemetry that exposes its SLIs. A service without observability is, for operational purposes, a service that does not exist.

RULE OPS-007 No incident MAY be closed without a published postmortem that identifies root cause, contributing factors, and accepted action items with owners and dates.

RULE OPS-008 No secret MAY be logged. The logging layer SHALL redact known secret

patterns, and the secret store SHALL be the single source of secret values; logs SHALL contain only secret references, never values.

3.3.6 Process Rules

RULE PROC-001 No implementation MAY proceed without a corresponding clause in the Engineering Constitution. Implementation that outruns specification is forbidden.

RULE PROC-002 No feature MAY be considered complete on the basis of a passing build alone. Runtime certification with objective evidence is mandatory.

RULE PROC-003 No pull request MAY be merged without a passing review checklist, a passing CI gate, and a passing runtime smoke test in a production-equivalent environment.

RULE PROC-004 No review MAY be approved without the reviewer citing the Constitution clause and Playbook checklist item that the implementation satisfies.

RULE PROC-005 No build-agent output MAY be merged without human review. The build agent is an implementation tool, not a reviewer; its outputs are subject to the full review and certification regime.

RULE PROC-006 No production release MAY proceed unless ALL of the following are satisfied: Engineering Constitution, Engineering Playbook, Runtime Certification, Production Acceptance Criteria. This is the Production Acceptance Gate and it admits no override below the Chief Systems Architect.

RULE PROC-007 No discrepancy between static analysis and runtime behaviour MAY be resolved by suppressing the runtime evidence. The discrepancy SHALL be resolved by changing the implementation or the specification, and the resolution SHALL be recorded.

RULE PROC-008 No engineering document MAY be considered final without a named owner, a version, a last-reviewed date, and a next-review date.

RULE PROC-009 No exception to any rule in this chapter MAY be granted verbally. Exceptions SHALL be recorded in the exceptions register with a sponsor, a justification, a sunset date, and a remediation plan.

RULE PROC-010 No rule in this chapter MAY be silently reinterpreted. Reinterpretation SHALL be raised as a governance proposal and resolved through the Chapter 4.16 governance process before it takes effect.

3.4 Exceptions Register

An exceptions register SHALL be maintained by the Office of the Chief Systems Architect and SHALL be readable by every member of the engineering organization. Each entry SHALL contain: the rule identifier (for example, EXT-002), the sponsor, the justification, the sunset date, the remediation plan, and the status. Exceptions are not a relaxation of the rules; they are a tracked, time-bounded, publicly visible acknowledgement that a specific rule cannot yet be satisfied, with a committed path to satisfaction. An exception that passes its sunset date without remediation SHALL be escalated to the Chief Systems Architect for resolution.

Chapter 4 — Required Constitution Chapters

The Engineering Constitution is the formal specification of the Opus Publica platform. It is normative: every implementer, every reviewer, and every build agent is bound by it, and every implementation decision is traceable to a clause within it. This chapter enumerates the chapters that the Engineering Constitution **MUST** contain. Each entry below specifies the chapter's purpose, its mandatory scope, and its acceptance criteria — the conditions under which the Constitution chapter itself is considered complete. The Engineering Leadership Team **SHALL** author each chapter, **SHALL** assign it a named owner, and **SHALL** sign it off before any implementation of the corresponding platform capability begins.

Every Constitution chapter **SHALL** follow the documentation standard defined in Chapter 6 of this Directive: Purpose, Responsibilities, Inputs, Outputs, State Transitions, Error Conditions, Recovery, Runtime Evidence, and Acceptance Criteria. The acceptance criteria specified in this chapter are the Constitution-level criteria; each Constitution chapter **SHALL**, in addition, specify the implementation-level acceptance criteria that the Playbook will enforce.

4.01 Architecture

Mandatory Scope. Specify the service decomposition, the service boundaries, the communication patterns (synchronous RPC, asynchronous messaging, event streaming), the deployment topology, and the technology stack decisions. Document the mapping to Conway's Law (Principle 11).

Acceptance Criteria. A published service map; a published service-interface catalogue; an Architecture Decision Record for each non-obvious decision; a Conway mapping of service to owning team.

4.02 Data Model

Mandatory Scope. Specify the canonical data model for every mastered datum: articles, authors, identifiers, roles, journals, issues, submissions, reviews, decisions, versions, preservation records, audit records. Specify the single source of truth for each datum.

Acceptance Criteria. An entity-relationship diagram; a data dictionary with one entry per mastered datum naming its owning service; a denormalization register listing every derived copy with its source and invalidation strategy.

4.03 Workflow

Mandatory Scope. Specify the editorial state machine: states, transitions, preconditions, postconditions, side effects, and idempotency contracts. Cover submission, assignment, review, decision, revision, copyedit, production, publication, correction, retraction.

Acceptance Criteria. A state-machine diagram; a transition table enumerating every legal transition with its preconditions and side effects; a machine-readable specification against which runtime transitions are validated.

4.04 Rendering

Mandatory Scope. Specify how the canonical JATS representation is rendered to HTML, PDF, citations, and other formats. Specify the rendering pipeline, the template system, the determinism requirements, and the content-addressing of rendered artifacts.

Acceptance Criteria. A rendering pipeline diagram; a determinism proof (or test) showing that identical JATS input produces byte-identical output; a content-addressing specification.

4.05 Storage

Mandatory Scope. Specify the storage layer: object storage for artifacts, relational storage for mastered data, log storage for the audit log, cache storage for derived values. Specify replication, retention, and durability targets.

Acceptance Criteria. A storage map; a retention schedule per datum class; a durability SLO per storage tier; a backup and restore procedure with tested restore times.

4.06 Security

Mandatory Scope. Specify the security model: threat model, attack surface, defence in depth, secret management, network policy, dependency vulnerability management, and incident response.

Acceptance Criteria. A published threat model; a secret-management specification; a dependency-vulnerability policy with SLAs for patching; an incident-response runbook.

4.07 Authentication

Mandatory Scope. Specify the authentication model: identity providers, session management, token formats, token lifetimes, refresh policies, and multi-factor requirements. Cover user, service, and build-agent identities.

Acceptance Criteria. An authentication flow diagram; a token specification; a session-management specification; an MFA policy.

4.08 Authorization

Mandatory Scope. Specify the authorization model: roles, permissions, role-permission mappings, object-level access control, and the enforcement points. Cover the journal, section, and article scopes.

Acceptance Criteria. A role-permission matrix; an object-level access control specification; an enforcement point catalogue; a test matrix proving every permission boundary is enforced.

4.09 Publication

Mandatory Scope. Specify the publication event: preconditions, atomic operations, side effects (DOI minting, Crossref deposit, preservation deposit, OAI-PMH update, ORCID write-back), and the post-publication state.

Acceptance Criteria. A publication-event specification; an outbox specification for each side effect; a transactional guarantee that the publication event and its outbox events commit atomically.

4.10 DOI

Mandatory Scope. Specify the DOI lifecycle: assignment, minting via Crossref or DataCite, deposit, resolution, content negotiation, update, and the relationship to versions. Specify the DOI registry table.

Acceptance Criteria. A DOI-lifecycle state machine; a DOI registry schema; a content-negotiation specification; a test proving DOIs are never reused or reassigned.

4.11 Crossref

Mandatory Scope. Specify the Crossref integration: deposit API (test and live), REST API as client and server, schema version, webhook handling, reference matching, and the Similarity Check integration. Specify the idempotency and reconciliation model.

Acceptance Criteria. A Crossref integration specification; a deposit-XML schema pinned to a Crossref schema version; a reconciliation job specification; an idempotency-key specification.

4.12 Artifacts

Mandatory Scope. Specify the artifact model: what constitutes an artifact, how it is named, how it is content-addressed, how it is stored, how it is retrieved, and how its lifecycle (including deletion for legal reasons under governed process) is managed.

Acceptance Criteria. An artifact taxonomy; a content-addressing specification (SHA-256); a retrieval interface; a lifecycle specification including the governed deletion procedure.

4.13 Templates

Mandatory Scope. Specify the template system: what is templated (article HTML, PDF, email, citation, landing page), how templates are versioned, how they are validated, and how they are rendered deterministically.

Acceptance Criteria. A template catalogue; a versioning specification; a validation specification; a determinism test proving identical inputs produce identical outputs.

4.14 Testing

Mandatory Scope. Specify the testing strategy: unit, integration, contract, end-to-end, property-based, runtime invariant, chaos, and load. Specify the test pyramid, the coverage targets, and the gating policy.

Acceptance Criteria. A testing strategy document; a test pyramid specification; a coverage target per layer; a gating policy specifying which tests block merge and which block release.

4.15 Certification

Mandatory Scope. Specify the certification regime: the five verification pillars (runtime, screenshots, database, storage, API), the certification artifact set, the sign-off process, and the re-certification triggers.

Acceptance Criteria. A certification specification; an artifact-set checklist; a sign-off process; a re-certification trigger catalogue.

4.16 Governance

Mandatory Scope. Specify the governance model: how the Constitution and Playbook are amended, who approves amendments, how precedents are recorded, and how exceptions are granted and sunset.

Acceptance Criteria. A governance process specification; an amendment workflow; a precedent register; an exceptions register.

4.17 Versioning

Mandatory Scope. Specify the versioning model: platform versioning, schema versioning, API versioning, artifact versioning, and document versioning. Specify the compatibility contract for each.

Acceptance Criteria. A versioning specification per layer; a compatibility matrix; a deprecation policy.

4.18 Migration

Mandatory Scope. Specify the migration model: schema migrations, data migrations, OJS-to-Opus-Publica migrations, and platform-version migrations. Specify idempotency, reversibility, and the migration testing regime.

Acceptance Criteria. A migration framework specification; a migration testing regime; an OJS-import specification with a documented fidelity map.

4.19 Observability

Mandatory Scope. Specify the observability model: metrics, logs, traces, and events. Specify the SLI / SLO / error-budget framework, the dashboard catalogue, and the alerting catalogue.

Acceptance Criteria. An observability specification; an SLI/SLO catalogue per service; an error-budget policy; a dashboard and alert catalogue.

4.20 Preservation

Mandatory Scope. Specify the preservation model: CLOCKSS, LOCKSS, Portico, and PKP PN as pluggable backends behind a common contract. Specify deposit, receipt, trigger-event monitoring, and PREMIS metadata.

Acceptance Criteria. A preservation contract specification; a backend catalogue; a deposit specification per backend; a PREMIS metadata specification.

4.21 Indexing and Harvesting

Mandatory Scope. Specify the indexing and harvesting interfaces: OAI-PMH (oai_dc, OpenAIRE, JATS), SWORD v3 (server and client), DOAJ export, OpenAIRE compliance, and COUNTER 5.1.1 event recording (SUSHI serving is RC2).

Acceptance Criteria. An interface specification per protocol; a compliance matrix; a test suite proving compliance with each protocol's conformance requirements.

4.22 Identifiers

Mandatory Scope. Specify the identifier model: DOI, ORCID, PII, PMID, arXiv, handle, ISBN, ISSN, UUID, publisher-id. Specify the typed-identifier set per article, format validation per type, and inbound normalization.

Acceptance Criteria. An identifier taxonomy; a format-validation specification per type; a normalization specification; a test suite covering valid and invalid identifiers per type.

4.23 Peer Review and Editorial Model

Mandatory Scope. Specify the peer-review model: single-blind, double-blind, open. Specify reviewer assignment, conflict-of-interest declarations, anonymization at the platform layer, reviewer confidentiality, and COPE alignment.

Acceptance Criteria. A review-model specification; an anonymization specification (including a test proving author-identifying metadata is stripped in double-blind mode); a COPE compliance matrix.

4.24 Integration and External Services

Mandatory Scope. Specify the catalogue of external services the platform integrates with, the integration pattern for each, the credential management, the retry and circuit-breaker policy, and the reconciliation model.

Acceptance Criteria. An integration catalogue; a pattern specification per integration; a credential map (references only, never values); a reconciliation specification.

4.25 Constitution Completion Criteria

The Engineering Constitution is considered complete for RC1 when every chapter listed above (4.01 through 4.24) is authored, reviewed, signed off by its named owner, and cross-referenced from every other chapter that depends on it. A chapter in isolation is not sufficient: the Constitution is a web of cross-references, and a missing cross-reference is a defect. The Release Certification Authority SHALL verify cross-reference completeness as part of RC1 certification.

Chapter 5 — Required Playbook Chapters

The Engineering Playbook is the operationalization of the Engineering Constitution. Where the Constitution specifies what the platform is, the Playbook specifies how the platform is built, reviewed, tested, certified, and released. The Playbook is normative for every implementer, every reviewer, every build agent, and every operator. It is the document that converts the Constitution's requirements into checklists, runbooks, and gates that can be applied at the moment of implementation.

This chapter enumerates the chapters that the Engineering Playbook **MUST** contain. Each entry specifies the chapter's purpose, its mandatory scope, and its acceptance criteria. As with the Constitution, every Playbook chapter **SHALL** follow the documentation standard in Chapter 6 of this Directive.

5.01 Implementation Lifecycle

Mandatory Scope. Specify the end-to-end lifecycle of an implementation: from Constitution clause selection, through task definition, implementation, review, certification, release, and post-release ownership. Specify the entry and exit criteria for each stage.

Acceptance Criteria. A lifecycle diagram; a stage-by-stage entry/exit criteria table; a checklist per stage.

5.02 Branching and Version Control

Mandatory Scope. Specify the branching model (trunk-based, GitFlow, or other), the branch naming convention, the merge strategy, the commit-message convention, and the rules for build-agent branches. Specify what is version-controlled (code, configs, prompts, migrations, docs) and what is not.

Acceptance Criteria. A branching-model specification; a naming convention; a commit-message convention; a register of version-controlled artifact classes.

5.03 Commit and Pull-Request Conventions

Mandatory Scope. Specify the commit granularity, the pull-request description template, the required metadata (Constitution clause reference, Playbook checklist reference, risk classification), and the size limits above which a pull request must be split.

Acceptance Criteria. A pull-request template; a metadata specification; a size-limit policy with splitting guidance.

5.04 Code Review

Mandatory Scope. Specify the code-review regime: who reviews, how many reviewers are required for each risk class, what the review checklist contains, how review comments are classified (blocking, non-blocking, nit), and the SLA for review turnaround. Specify the rules for build-agent-authored code.

Acceptance Criteria. A review-regime specification; a review checklist (one item per Playbook rule); a comment-classification specification; a build-agent review specification.

5.05 Quality Assurance

Mandatory Scope. Specify the QA regime: the test layers, the test ownership (who writes which tests), the test environment topology, the test-data management strategy, and the gating policy. Specify the runtime-invariant monitoring regime.

Acceptance Criteria. A QA-regime specification; a test-environment topology; a test-data specification; a runtime-invariant monitoring catalogue.

5.06 Certification Process

Mandatory Scope. Specify the certification process: who certifies, what the certification artifact set is, how artifacts are stored and referenced, how sign-off is recorded, and how re-certification is triggered. Specify the role of the Release Certification Authority.

Acceptance Criteria. A certification-process specification; an artifact-set checklist; a sign-off specification; a re-certification trigger catalogue.

5.07 Release and Deployment

Mandatory Scope. Specify the release regime: release cadence, release naming, the deployment pipeline, the staged rollout strategy, the production-readiness review, and the rollback procedure. Specify the rules for emergency releases.

Acceptance Criteria. A release-regime specification; a deployment-pipeline specification; a staged-rollout specification; a rollback runbook; an emergency-release specification.

5.08 Rollback and Incident Response

Mandatory Scope. Specify the rollback procedure: when to roll back, who has authority, how the rollback is tested, and how the post-rollback verification is performed. Specify the incident-response regime: severity levels, escalation, communication, and postmortem.

Acceptance Criteria. A rollback runbook; a post-rollback verification checklist; an incident-response specification; a postmortem template.

5.09 On-Call and Operational Ownership

Mandatory Scope. Specify the on-call regime: rotation, handoff, escalation, authority to roll back, and the relationship between on-call and the owning team. Specify the rules for services without an on-call rotation (they cannot be released).

Acceptance Criteria. An on-call-regime specification; a handoff protocol; an escalation path; a no-on-call-no-release rule.

5.10 Documentation Discipline

Mandatory Scope. Specify the documentation regime: what must be documented, where it lives, how it is versioned, how it is reviewed, and how it is kept current. Specify the relationship between documentation and the certification artifact set.

Acceptance Criteria. A documentation-regime specification; a documentation catalogue; a review cadence; a staleness-detection mechanism.

5.11 Playbook Completion Criteria

The Engineering Playbook is considered complete for RC1 when every chapter listed above (5.01 through 5.10) is authored, reviewed, signed off by its named owner, and is demonstrably applicable — meaning that at least one implementation, review, certification, and release cycle has been executed against each chapter and the chapter has been found to be unambiguous, complete, and enforceable. A Playbook chapter that has never been applied is, by definition, a draft, regardless of how thoroughly written.

5.12 The Review Checklist Contract

Every rule in Chapter 3 (Non-Negotiable Rules) and every principle in Chapter 2 (Engineering Philosophy) SHALL appear as a discrete, checkable item in the Playbook's review checklist. The checklist SHALL be machine-readable (YAML or JSON) so that it can be enforced by CI as well as by human reviewers. A pull request SHALL not be mergeable until every checklist item is either marked satisfied (with a pointer to the evidence) or marked not-applicable (with a justification). The checklist is the operational bridge between the Constitution and the implementation; its completeness is non-negotiable.

Chapter 6 — Documentation Standards

Every chapter of the Engineering Constitution and Engineering Playbook SHALL follow the documentation standard specified in this chapter. The standard is not a style preference; it is the structure that makes a chapter reviewable, certifiable, and operable. A chapter that omits any of the nine mandatory sections is, by definition, incomplete and SHALL NOT be signed off. The nine sections exist because each answers a question that an implementer, a reviewer, or an operator will ask; omitting a section leaves that question unanswered and forces the reader to infer the answer, which is precisely the kind of assumption that Principle 5 (Evidence Before Assumption) forbids.

6.1 The Nine Mandatory Sections

Each Constitution chapter and each Playbook chapter SHALL contain the following nine sections, in the following order, with the following content. The section headings MAY be adapted to the chapter's domain, but the section's content obligation MAY NOT be omitted.

1. Purpose

A concise statement of what the chapter specifies and why it exists. The Purpose section SHALL answer: what problem does this chapter solve, whose problem is it, and what happens if this chapter did not exist? The Purpose section SHALL be no longer than 200 words and SHALL be readable in isolation by a new engineering leader.

2. Responsibilities

A statement of who owns the chapter, who is accountable for its implementation, who is responsible for its execution, and who is consulted and informed. The Responsibilities section SHALL use the RACI model (Responsible, Accountable, Consulted, Informed) and SHALL name a single Accountable owner; shared accountability is forbidden because shared accountability is no accountability.

3. Inputs

An enumeration of everything the chapter's subject consumes: upstream data, configuration, external service responses, user inputs, and triggers. Each input SHALL be named, typed, sourced (which service or user provides it), and constrained (what values are valid). Inputs that are not enumerated are, by Principle 7 (No Hidden State), forbidden.

4. Outputs

An enumeration of everything the chapter's subject produces: downstream data, rendered artifacts, external service deposits, notifications, and audit records. Each output SHALL be named, typed, destined (which service or user consumes it), and contracted (what guarantees the output makes). Outputs that are not enumerated cannot be certified.

5. State Transitions

A specification of every state transition the chapter's subject performs or observes. Each transition SHALL be named, shall have its preconditions and postconditions specified, shall have its side effects enumerated, and shall have its idempotency contract stated. Where the chapter's subject has no state transitions (for example, a pure rendering function), this section SHALL state so explicitly rather than be omitted.

6. Error Conditions

An enumeration of every error condition the chapter's subject can encounter, classified by recoverability (transient, permanent, requires-human). Each error condition SHALL be named, shall have its detection specified (what signal indicates the condition), and shall have its user-visible message specified. Undocumented error conditions are the proximate cause of most operational incidents and are forbidden.

7. Recovery

A specification of the recovery procedure for each error condition. Each recovery SHALL be named, shall specify whether it is automatic or manual, shall specify the actor who performs it, and shall specify the verification that confirms recovery. Where recovery is not possible (a permanent error), this section SHALL specify the escalation path.

8. Runtime Evidence

A specification of the runtime evidence that demonstrates the chapter's subject is operating correctly. This section SHALL enumerate the SLIs, the audit-log entries, the invariant monitors, and the dashboard panels that constitute the evidence. The section SHALL be the basis on which Runtime Certification (Chapter 8) is performed; a chapter without a Runtime Evidence section cannot be certified.

9. Acceptance Criteria

A specification of the conditions under which the chapter's implementation is considered complete. Each criterion SHALL be testable, shall reference the Runtime Evidence that demonstrates it, and shall be binary (satisfied or not satisfied; “mostly working” is not an acceptance state). The Acceptance Criteria section SHALL be the contract between the chapter's author and the Release Certification Authority.

6.2 Documentation Conventions

Beyond the nine mandatory sections, every chapter SHALL observe the following conventions. These conventions exist to make the Constitution and Playbook machine-parseable, cross-referenceable, and diffable across versions.

- **Requirement Identifiers.** Every requirement statement SHALL use RFC 2119 vocabulary (MUST, SHALL, SHOULD, MAY) and SHALL be uniquely identifiable (for example, PUB-001, ARCH-003).

- **Cross-References.** Every cross-reference SHALL be a stable identifier, not a page number or a section title that may change. Cross-references SHALL be bidirectional where the relationship is bidirectional.
- **Diagrams.** Every diagram SHALL have a caption, a number, and a textual description sufficient to understand the diagram without the visual. Diagrams SHALL be authored in a version-controllable format (Mermaid, PlantUML, or source SVG), never as binary images.
- **Code Examples.** Every code example SHALL specify the language, SHALL be runnable, and SHALL be tested as part of the documentation build. Examples that do not run are forbidden because they are unverified claims.
- **Chapter Headers.** Every chapter SHALL carry a header identifying its owner, its version, its last-reviewed date, and its next-review date. A chapter without a next-review date is assumed stale.
- **Voice and Tense.** Every chapter SHALL be written in the active voice and the present tense. The passive voice obscures the actor and is forbidden in requirement statements.

6.3 Diagram Catalogue

The following diagrams are mandatory artifacts of the Constitution and SHALL be maintained alongside the chapters that own them. Each diagram SHALL be regenerated from its source on every documentation build, so that the diagram in the document is always the diagram the source produces.

Diagram	Owning Chapter	Purpose
Service Map	4.01 Architecture	Shows every service, its boundaries, and its synchronous and asynchronous dependencies.
Editorial State Machine	4.03 Workflow	Shows every editorial state and every legal transition, with side effects.
Publication Event Flow	4.09 Publication	Shows the atomic publication event and its outbox side effects.
DOI Lifecycle	4.10 DOI	Shows the DOI states from assignment through resolution and update.
Rendering Pipeline	4.04 Rendering	Shows the path from canonical JATS to rendered HTML, PDF, and citations.
Preservation Deposit Flow	4.20 Preservation	Shows the deposit to each preservation backend and the receipt handling.
Identifier Topology	4.22 Identifiers	Shows the typed-identifier set per article and the relationships among them.

Authorization Matrix	4.08 Authorization	Shows the role-permission-object triples that constitute the access model.
Observability Topology	4.19 Observability	Shows the metrics, logs, traces, and events flow from each service.
Conway Mapping	4.01 Architecture	Shows the service-to-team mapping that satisfies Principle 11.

Chapter 7 — Coding Standards

Coding standards are the lowest-level operationalization of the engineering philosophy. This chapter specifies the coding standards that the Engineering Playbook SHALL enforce on every line of code merged into the Opus Publica codebase, whether authored by a human or by a build agent. The standards are organized as obligations (what MUST be done) and prohibitions (what MUST NOT be done), and each is paired with the review-checklist question that detects a violation.

7.1 Single Implementation

Every piece of domain logic SHALL have exactly one implementation. Where two implementations appear to be needed (for example, a server-side and a client-side rendering of the same citation), the duplication SHALL be eliminated by extracting the shared logic into a single module that both contexts consume. The Playbook SHALL require that any pull request introducing a function whose name or purpose duplicates an existing function be rejected, and that the duplication be resolved by abstraction before merge. Single implementation is the operational form of Principle 2 (No Duplicate Logic); its violation is the most common source of the divergence bugs that characterize the implementation loop.

- **Review checklist.** Does this pull request introduce a function whose name or purpose duplicates an existing function? If yes, the duplication SHALL be eliminated before merge.

7.2 Single Ownership

Every module, every service, and every shared utility SHALL have a single owning team. Ownership is recorded in a CODEOWNERS file that the Playbook SHALL require be kept current. A pull request that modifies a module SHALL require approval from the module's owning team; a pull request that modifies a module with no declared owner SHALL be rejected until ownership is declared. Ownership is not stewardship: the owning team carries the pager, performs the reviews, and is accountable for the module's behaviour in production. Shared ownership of a module is forbidden because it produces the tragedy-of-the-commons pattern in which no team invests in the module's long-term health.

- **Review checklist.** Does the modified module have a declared owner in CODEOWNERS? Does the pull request carry the owning team's approval?

7.3 No Duplicated Utilities

Utility functions — date formatting, identifier validation, string normalization, error formatting, and the like — SHALL live in a single shared utilities package with a declared owner. The temptation to write a small helper inline because “it is just one function” SHALL be resisted: that one function becomes ten functions across ten files within a release, and the divergence begins. The Playbook SHALL require that any new utility function be added to the shared package, not inlined, and that any inlined utility be extracted during review. The shared utilities package SHALL itself be versioned, with a deprecation policy for removed or renamed utilities.

- **Review checklist.** Does this pull request inline a utility that should live in the shared utilities package? If yes, the utility SHALL be extracted before merge.

7.4 No Magic Strings

String literals that carry domain meaning — state names, event names, role names, identifier types, error codes, configuration keys — SHALL be defined as named constants in a single, typed registry, and SHALL be referenced by name everywhere they are used. A string literal that appears in more than one location is, by definition, a magic string, and is a divergence bug waiting to happen: when the constant's value changes, the magic string does not, and the divergence is silent until a runtime failure exposes it. The Playbook SHALL require that linters detect repeated string literals above a threshold and that the threshold be enforced as a merge gate.

This rule applies with particular force to editorial state names, identifier type names (doi, orcid, pii, pmid, arxiv), and Crossref metadata field names. These are the vocabulary of the platform, and a vocabulary that drifts is a platform that drifts. The Constitution SHALL publish the platform's vocabulary as a typed enumeration, and the Playbook SHALL forbid any code that uses a vocabulary term not present in the enumeration.

- **Review checklist.** Does this pull request introduce a string literal that carries domain meaning? If yes, the literal SHALL be replaced with a named constant from the vocabulary registry before merge.

7.5 No Fallback Hacks

Defensive code that silently substitutes a default value when an expected value is absent — “if (doi == null) doi = ‘unknown’” — SHALL be treated as a defect, not as robustness. A fallback hack masks a bug in the upstream code that should have produced the value, and the masking propagates: the fallback value is consumed downstream as if it were real, and the discrepancy is discovered only when a Crossref deposit fails or a reader reports a broken citation. The Playbook SHALL require that any code path that encounters an unexpected null or missing value fail loudly, with an audit-log entry and an alert, rather than silently substitute a default. Loud failure is the precondition for evidence; silent fallback is the precondition for the implementation loop.

The narrow exception is user-facing input validation, where a missing value is expected and the correct response is a validation error returned to the user. That is not a fallback hack; it is a contract enforcement. The distinction is whether the missing value is expected (contract enforcement, permitted) or unexpected (a bug, which must fail loudly). The Playbook SHALL provide examples of both and SHALL train reviewers to distinguish them.

- **Review checklist.** Does this pull request introduce a fallback that silently substitutes a default for an unexpected missing value? If yes, the fallback SHALL be replaced with a loud failure before merge.

7.6 Type Discipline

Every language in the platform SHALL be used with its strongest available type system. TypeScript SHALL be used in strict mode with no implicit any and no unchecked casts. Python SHALL be used with full type hints and a runtime type checker on service boundaries. SQL SHALL be used through a typed query layer that prevents injection and validates results. The type system is the first line of defence against the implementation loop: a type error caught at compile time is a bug caught before it reaches runtime, and a bug caught before runtime is a bug that does not require runtime certification to discover. The Playbook SHALL forbid the suppression of type errors except with a recorded justification, and SHALL require that suppressed type errors be tracked to resolution.

- **Review checklist.** Does this pull request suppress a type error? If yes, the suppression SHALL carry a recorded justification and a tracking issue before merge.

7.7 Error Handling

Errors SHALL be handled at the layer that has the context to handle them, and SHALL be propagated, with enrichment, to that layer. An error swallowed at a low layer is an error invisible at every higher layer, and invisibility is the precondition for the implementation loop. The Playbook SHALL specify the platform's error-handling taxonomy: transient errors (retryable), permanent errors (not retryable, surface to user), and invariant errors (the system is in an impossible state, page on-call). Each error class SHALL have a defined handling pattern, a defined log entry, and a defined user-visible message. Ad hoc error handling is forbidden.

- **Review checklist.** Does this pull request swallow an error? If yes, the error SHALL be propagated or handled per the platform's error-handling taxonomy before merge.

7.8 Testing Conventions

Every code change SHALL be accompanied by tests at the appropriate layer. Unit tests SHALL cover the logic of the changed module; integration tests SHALL cover the module's interaction with its declared dependencies; contract tests SHALL cover the module's conformance to its published interface; and end-to-end tests SHALL cover the user-visible behaviour the change affects. The Playbook SHALL specify the test pyramid and the coverage targets per layer, and SHALL forbid the merge of a pull request that reduces coverage below the target without a recorded justification. Runtime invariant monitors (Principle 6) SHALL be treated as tests and SHALL be maintained with the same discipline.

- **Review checklist.** Does this pull request include tests at every layer the change affects? Does it maintain or increase coverage at each layer?

7.9 Naming

Names SHALL be precise, consistent, and aligned with the platform's vocabulary registry (Rule 7.4). A function named `getArticle` that returns a submission is a lie, and lies in code are the precondition for the

implementation loop. The Playbook SHALL publish a naming convention per language and per layer, and SHALL forbid abbreviations except those explicitly registered. Consistency of naming is what permits a reader to navigate the codebase by expectation; inconsistency forces the reader to re-derive the meaning of every name, and the re-derivation is where assumptions enter.

- **Review checklist.** Does this pull request introduce a name that is inconsistent with the platform's naming convention or vocabulary registry? If yes, the name SHALL be corrected before merge.

7.10 Dependency Discipline

Every external dependency SHALL be pinned to a specific version, SHALL be reviewed for licence compatibility, shall be reviewed for security posture, and SHALL have a declared owner responsible for its upgrade. The Playbook SHALL forbid the introduction of a new dependency without an Architecture Decision Record justifying the choice over existing dependencies and over implementation in-platform. Transitive dependencies SHALL be audited on a regular cadence; a vulnerable transitive dependency is a vulnerability in the platform, regardless of whether the platform directly uses it.

- **Review checklist.** Does this pull request introduce a new dependency? If yes, is there an ADR justifying the choice, and is the dependency pinned, licensed, and owned?

Chapter 8 — Certification Standards

Certification is the gate at which runtime evidence is converted into a release decision. This chapter specifies what “complete” means for a feature of Opus Publica, what evidence SHALL be produced to demonstrate completeness, who produces the evidence, who verifies it, and how the verification is recorded. Certification is not QA; QA is the regime that produces the evidence, certification is the regime that verifies the evidence and authorizes release. The two are separated by design, because the engineer who wrote the feature is not the right person to certify it, and the engineer who ran the tests is not the right person to authorize the release.

8.1 Definition of “Complete”

Definition

A feature is complete when, and only when, it has been verified against ALL of the five certification pillars — Runtime, Visual, Database, Storage, and API — with objective evidence that has been reviewed and signed off by the Release Certification Authority. A feature that passes four of the five pillars is not “mostly complete”; it is incomplete, and SHALL NOT be released. There is no partial credit in certification, because there is no partial correctness in production.

8.2 The Five Certification Pillars

Each pillar answers a distinct question about the feature's behaviour. A feature that passes static analysis but has not been verified against all five pillars has not been verified at all; it has been assumed. The Principle of Runtime Truth (Foundational Principle I) makes this explicit: the gap between “it builds” and “it works in production” is exactly the gap that the five pillars are designed to close.

Pillar	Question Answered	Required Evidence
P1 — Runtime	Does the feature execute correctly in a production-equivalent environment, exercising the real code paths, the real external integrations, and the real data?	Runtime logs showing the state transitions; invariant monitor outputs showing no invariant violations; a recorded execution trace.
P2 — Visual (Screenshots)	Does the feature render correctly for the user, across the supported browsers, viewport sizes, and accessibility configurations?	Screenshots of every user-visible state of the feature, captured in a production-equivalent environment, with captions identifying the state.
P3 — Database	Does the feature produce the correct persisted state, in the	Database queries (read-only, against a replica) showing the

	correct location, with the correct relationships, and without corrupting unrelated state?	persisted state before and after the feature execution; verification that no unrelated rows changed.
P4 — Storage	Does the feature produce the correct artifacts in storage, content-addressed, with the correct metadata, and retrievable by the correct keys?	Storage listings showing the artifacts produced; SHA-256 hashes of the artifacts; retrieval tests proving the artifacts are fetchable by their content address.
P5 — API	Does the feature expose and consume APIs correctly — idempotent, versioned, authenticated, authorized, and conformant to the published contract?	API call recordings showing the request and response; contract-test results proving conformance; idempotency-test results proving retries are safe.

8.3 The Certification Artifact Set

For every feature certified, the following artifact set SHALL be produced, stored in the certification repository, and referenced from the release record. The artifact set is the evidence that the Release Certification Authority reviews; without the artifact set, there is no certification, and without certification, there is no release. The artifact set is itself immutable once certified: a certified feature's evidence does not change, because the certified feature does not change.

17. The Constitution clause and Playbook checklist item the feature implements, by stable identifier.
18. The runtime execution log, captured in a production-equivalent environment, showing the feature's state transitions.
19. The invariant monitor outputs, showing no invariant violations during the execution.
20. The visual evidence (screenshots or screen recordings) of every user-visible state of the feature.
21. The database verification queries and their results, showing the persisted state before and after.
22. The storage verification listing and hashes, showing the artifacts produced and their content addresses.
23. The API verification recordings, showing request/response pairs and contract-test results.
24. The idempotency test results, showing that retries produce the same end-state.
25. The rollback test results, showing that the feature can be rolled back without data loss.
26. The sign-off record, naming the certifying authority, the date, and the release the certification authorizes.

8.4 The Sign-Off Process

Certification sign-off is performed by the Release Certification Authority (RCA), a role distinct from the implementing team and distinct from QA. The RCA reviews the artifact set, verifies that each artifact is

present and complete, verifies that the evidence supports the certification claim, and either signs off (authorizing release) or rejects (returning the feature to the implementing team with specific deficiencies). The RCA SHALL NOT sign off a feature whose artifact set is incomplete; the RCA SHALL NOT sign off a feature whose runtime evidence contradicts the static analysis; the RCA SHALL NOT sign off a feature for which a Production Acceptance Criterion is unsatisfied. The RCA's sign-off is recorded in the release record and is itself audited.

The RCA is empowered to halt a release at any point up to deployment, and SHALL halt the release if any evidence emerges between sign-off and deployment that the feature is not, in fact, complete. The RCA is not empowered to override a certification failure; only the Chief Systems Architect may consider an exception, and only through the exceptions register (Chapter 3.4), and never for a Runtime Truth violation. This asymmetry is deliberate: the people closest to the evidence have the authority to stop, and the people farther from the evidence do not have the authority to override.

8.5 Re-Certification Triggers

A certified feature SHALL be re-certified when any of the following triggers fires. Re-certification is not a punishment; it is the acknowledgement that a feature's certification is valid only for the environment, dependencies, and specification against which it was originally certified, and that a change to any of those invalidates the certification.

- The feature's implementation is modified (any change to the code path the feature exercises).
- A dependency of the feature is upgraded to a new major version.
- The Constitution clause the feature implements is amended.
- The Playbook checklist the feature satisfies is amended.
- A runtime invariant the feature is bound by is added or modified.
- An incident postmortem identifies the feature as a contributing factor.
- The feature has not been re-certified within the maximum certification interval (defined by the Playbook; default twelve months).

8.6 The Certification Repository

All certification artifacts SHALL be stored in a dedicated certification repository, version-controlled, append-only for certified artifacts, and accessible to the RCA, Engineering Leadership, and external auditors. The certification repository is the platform's evidence store; it is the substrate on which the audit log (Principle 9) and the runtime verification (Principle 6) ultimately rest. A feature whose certification artifacts cannot be retrieved is, for audit purposes, a feature whose certification cannot be verified, and SHALL be treated as uncertified.

Chapter 9 — Future-Proofing

This chapter constrains how the platform may evolve in RC2, RC3, and beyond. It exists because the most expensive errors in platform engineering are not the errors of RC1 but the errors of RC1 that lock the platform into a shape it cannot escape in RC2 and RC3. The constraints below are not predictions; they are commitments that the RC1 Constitution SHALL make to the future versions of the platform, so that the future versions are not born constrained by RC1's accidents. Each constraint is paired with the RC1 design choice that honours it.

9.1 The RC2 / RC3 Roadmap

The release roadmap is structured as a sequence of release candidates, each of which expands the platform's contract surface while preserving the contracts established by its predecessors. RC1 establishes the engineering contract and the minimum viable publication platform. RC2 expands the platform's interoperability and tenancy. RC3 expands the platform's extensibility and intelligence. The boundaries between release candidates are not arbitrary; they correspond to the points at which a new contract surface can be added without destabilizing the existing surface.

Release	Theme	New Contract Surface	RC1 Constraint That Honours It
RC1	Contract	Engineering Constitution, Engineering Playbook, the seven definitional properties, the twenty-four Constitution chapters.	RC1 ships no surface it cannot certify; everything is specified before implementation.
RC2	Interoperability	Multi-tenant hosting, COUNTER 5.1.1 SUSHI serving, JATS round-trip authoring, expanded preservation backends.	RC1's preservation contract (Chapter 4.20) is pluggable, so RC2 adds backends without changing the contract.
RC3	Extensibility	Typed plugin SDK, plugin marketplace, AI-assisted editorial features, advanced XML tooling.	RC1's extension points are explicitly closed, so RC3 designs the SDK against a stable internal contract.

9.2 Plugin Architecture (RC3)

RC1 and RC2 ship no third-party plugin surface. The temptation to add ad hoc hooks during RC1 — “we will just expose this one internal function as a callback” — SHALL be resisted absolutely, because every

ad hoc hook added in RC1 becomes a constraint on the RC3 plugin SDK, and the SDK is then designed not against the platform the team wants but against the platform the team accidentally built. The RC1 Constitution SHALL explicitly enumerate the platform's extension points (currently: none) and SHALL forbid the addition of any extension point not on the list. The RC3 plugin SDK SHALL be designed against the stable RC2 internal contract, with a typed interface, a sandboxed execution model, and a versioning contract that permits plugin authors to target specific platform versions.

The plugin SDK SHALL learn from the OJS plugin model's failure: OJS plugins are coupled to internal call sites via a hook system, which means that every internal refactor breaks plugins, which in turn means that the platform cannot be refactored without breaking its ecosystem. The RC3 SDK SHALL expose plugins through a typed, versioned interface that is decoupled from internal implementation, so that the platform can be refactored without breaking plugins, and plugins can be versioned independently of the platform.

9.3 AI Integration (RC3)

AI-assisted editorial features — automated reviewer assignment suggestions, similarity-aware copyediting, citation normalization, abstract generation — are an RC3 concern. The RC1 Constitution SHALL not preclude them, but SHALL establish the constraints under which they will be evaluated when RC3 begins: every AI-assisted decision SHALL be auditable (the model, the version, the prompt, the inputs, the outputs, and the human decision that ratified or overturned the AI output SHALL be recorded); every AI-assisted decision SHALL be human-ratifiable (no AI output reaches the scholarly record without a human approval); every AI-assisted decision SHALL be reversible (a ratified AI output can be overturned by a later human decision, with the overturn recorded).

The Principle of Runtime Truth applies with full force to AI integration: an AI-assisted feature is not correct because the model produces output; it is correct when the runtime evidence demonstrates that the output, the human ratification, and the audit record all conform to the Constitution. AI features SHALL be certified against the same five pillars as any other feature, with the additional requirement that the audit trail for an AI-assisted decision SHALL be sufficient to reconstruct the decision after the fact, even if the model is later deprecated or replaced.

9.4 XML and JATS Evolution

The canonical article representation is JATS XML (ANSI/NISO Z39.96-2021). JATS is maintained by NISO with a published tag library and a versioned schema. The platform SHALL pin its JATS version per article (so that an article deposited under JATS 1.3 remains a JATS 1.3 article even after the platform upgrades to JATS 2.0), SHALL validate every JATS artifact against the official NLM schema at production time, and SHALL support round-trip ingestion of externally authored JATS. The Constitution SHALL forbid the silent upgrade of an article's JATS version; a version upgrade is a new version of the article, with all the versioning consequences that implies.

The platform SHALL track the evolution of JATS and SHALL adopt new versions through the governance process (Chapter 4.16), with a migration plan for existing articles (typically: leave them at their original version, and permit per-article opt-in upgrade with explicit versioning). The same discipline applies to other XML vocabularies the platform consumes or produces: Crossref deposit XML (pinned to a Crossref schema version), PREMIS, METS, and any future vocabularies adopted. XML version drift is a silent corruption of the scholarly record and is forbidden.

9.5 Crossref Evolution

The platform's Crossref integration SHALL be versioned against the Crossref schema library, which Crossref evolves over time. The Constitution SHALL specify the platform's schema-version policy (pin to a specific version, upgrade through governance), the deposit-test vs deposit-live endpoint discipline, the webhook handling for metadata-update notifications, and the reconciliation model that detects drift between the platform's view of a DOI and Crossref's view. The platform SHALL adopt new Crossref capabilities (Event Data, REST API extensions, new metadata fields) through the governance process, with a documented evaluation of the capability's value against the cost of adoption.

9.6 ORCID Evolution

The platform's ORCID integration SHALL support the ORCID public and member APIs, SHALL implement the OAuth 2.0 three-legged flow with secure token storage, SHALL support member-API write-back of works and peer-review records, and SHALL surface ORCID IDs in the JATS XML and Crossref deposits. The platform SHALL track ORCID API evolution (the current version is ORCID API 3.0) and SHALL adopt new versions through the governance process. The Constitution SHALL forbid the auto-population of ORCID data without user consent (Rule EXT-005), and SHALL specify the consent flow that permits write-back.

9.7 CLOCKSS, LOCKSS, and Portico

The platform's preservation model SHALL treat CLOCKSS, LOCKSS, Portico, and PKP PN as pluggable backends behind a common contract (Chapter 4.20). RC1 SHALL implement at least one backend; RC2 SHALL add the others. The common contract SHALL specify the deposit operation (idempotent, with receipt), the receipt handling (recorded, reconciled, alerted on failure), the trigger-event monitoring (operational concern), and the PREMIS metadata (Object, Event, Agent records per preserved artifact). The Constitution SHALL forbid the coupling of preservation logic to a specific backend's API; the backend is a pluggable component, and the platform's preservation contract is the abstraction over all of them.

The preservation deposit SHALL be queued as an outbox event at the moment of publication (Rule PUB-008), and SHALL be delivered at-least-once with an idempotent consumer. The deposit SHALL include the article's final published version, the JATS XML, and the supplements. The receipt SHALL be recorded and reconciled; a deposit whose receipt is not reconciled within the platform's SLO SHALL trigger an alert and an escalation. Preservation is not a best-effort operation; it is a contractual obligation to the scholarly record, and the Constitution SHALL treat it as such.

9.8 Backward Compatibility Contract

Each release candidate SHALL preserve the contracts established by its predecessors. The backward-compatibility contract has three layers: API compatibility (an API introduced in RC1 continues to behave identically in RC2 and RC3, or is deprecated with a documented window); data compatibility (data mastered in RC1 continues to be readable in RC2 and RC3, or is migrated with a documented migration); and artifact compatibility (an artifact published under RC1 continues to be retrievable, byte-identical, under RC2 and RC3, indefinitely). The last is the strongest: published artifacts are immutable (Principle 4) and their retrievability is guaranteed for the lifetime of the platform, regardless of release boundaries.

Where a breaking change is genuinely necessary — for example, a security fix that requires an incompatible API behaviour — the change SHALL be made through the governance process, with a documented deprecation window, a migration path, and a communication plan to affected consumers. Breaking changes without governance are forbidden; they are the engineering equivalent of silently overwriting a published article, and they corrode the trust on which the platform's adoption depends.

Chapter 10 — Deliverables

This chapter enumerates the documents that the Engineering Leadership Team SHALL produce, maintain, and sign off under this Directive. Each deliverable has a named owner, a defined audience, a defined review cadence, and a defined relationship to the other deliverables. A deliverable that is not maintained is, for certification purposes, a deliverable that does not exist; the RC1 certification SHALL verify the existence, currency, and sign-off of each deliverable before RC1 is declared.

10.1 The Eight Required Deliverables

#	Deliverable	Owner	Audience	Review Cadence
D1	Engineering Constitution	Chief Systems Architect	All engineers, reviewers, build agents	Quarterly, or on RC transition
D2	Engineering Playbook	Head of Engineering	All engineers, reviewers, build agents	Monthly, or on process change
D3	API Reference	Head of Platform Engineering	Implementers, integrators, external consumers	On every API change
D4	Architecture Guide	Principal Architect	Engineering Leadership, new engineers	Quarterly, or on architecture change
D5	Database Specification	Head of Data Engineering	Implementers, DBAs, auditors	On every schema change
D6	Deployment Guide	Head of Platform Engineering	Operators, on-call engineers	On every deployment-pipeline change
D7	QA Manual	Head of Quality Assurance	QA engineers, implementers	On every QA-regime change
D8	Release Certification Manual	Release Certification Authority	RCA, Engineering Leadership	On every certification-regime change

10.2 Deliverable Specifications

D1 — Engineering Constitution

The formal specification of the Opus Publica platform, comprising the twenty-four chapters enumerated in Chapter 4 of this Directive. The Constitution is normative: every implementation decision is traceable to a clause within it, and every clause is owned, versioned, and signed off. The Constitution is the document against which Runtime Certification is performed; a feature that cannot be traced to a Constitution clause is, by Rule PROC-001, an out-of-process feature and SHALL be halted.

D2 — Engineering Playbook

The operationalization of the Constitution, comprising the ten chapters enumerated in Chapter 5 of this Directive. The Playbook converts the Constitution's requirements into checklists, runbooks, and gates. The Playbook is the document the implementer, the reviewer, and the build agent consult at the moment of implementation; it is the day-to-day operating manual of the engineering organization.

D3 — API Reference

The complete, machine-readable specification of every API the platform exposes (internal and external), including request and response schemas, authentication and authorization requirements, idempotency contracts, versioning, error codes, and examples. The API Reference SHALL be generated from the API implementation's annotations, so that the reference and the implementation cannot drift. The Reference SHALL be the single source of truth for the API's contract; the implementation SHALL conform to it, not the reverse.

D4 — Architecture Guide

The narrative explanation of the platform's architecture, intended for a new engineer or engineering leader to understand the platform's shape, its service decomposition, its communication patterns, and its design decisions. The Architecture Guide SHALL include the service map, the Conway mapping, the Architecture Decision Records, and the rationale for each non-obvious decision. The Guide is the document that prevents architectural knowledge from being tacit and therefore lost when individuals leave.

D5 — Database Specification

The complete specification of every mastered datum, its owning service, its schema, its relationships, its migrations, and its retention. The Database Specification SHALL include the entity-relationship diagram, the data dictionary, the denormalization register, and the migration history. The Specification is the document against which Rule DATA-001 (single source of truth) and Rule DATA-002 (forward-compatible migrations) are enforced.

D6 — Deployment Guide

The complete specification of the deployment pipeline, the deployment topology, the staged rollout strategy, the rollback procedure, and the emergency-release procedure. The Deployment Guide SHALL be the document the on-call engineer consults during an incident; it SHALL be tested by actual deployment and rollback exercises, not merely written. A Deployment Guide that has never been exercised is a draft.

D7 — QA Manual

The complete specification of the QA regime: the test layers, the test ownership, the test environment topology, the test-data management strategy, the gating policy, and the runtime-invariant monitoring catalogue. The QA Manual is the document that operationalizes Principle 6 (Runtime Verification) and Chapter 8 (Certification Standards); it is the regime that produces the evidence the RCA verifies.

D8 — Release Certification Manual

The complete specification of the certification regime: the five pillars, the artifact set, the sign-off process, the re-certification triggers, and the certification repository. The Release Certification Manual is the document the RCA operates by; it is the regime that converts runtime evidence into a release decision. The Manual SHALL be the authoritative source for the Production Acceptance Gate.

10.3 Document Control

Every deliverable SHALL carry a document-control block specifying: the document identifier, the version, the owner, the last-reviewed date, the next-review date, the classification, and the change history. The document-control block is the metadata that permits the deliverable to be audited; a deliverable without a document-control block is, for audit purposes, unattributable and SHALL be treated as a draft. The change history SHALL record every amendment, its sponsor, its rationale, and its date, so that the evolution of the deliverable is itself auditable.

All deliverables SHALL be stored in version control alongside the codebase they govern, SHALL be reviewed through the same pull-request process as code, and SHALL be signed off by their named owner through a recorded approval. A deliverable that is not in version control is a deliverable that cannot be diffed, cannot be audited, and cannot be trusted; the Playbook SHALL forbid the existence of an out-of-version-control deliverable.

10.4 Deliverable Completion for RC1

RC1 is not declared complete until all eight deliverables exist, are version-controlled, are signed off by their named owners, and have been exercised against at least one end-to-end implementation, review, certification, and release cycle. The RC1 sign-off is performed by the Chief Systems Architect, on the recommendation of the Release Certification Authority, and is recorded in the release record with the evidence of each deliverable's existence, currency, and exercise. From the moment of RC1 sign-off, the Production Acceptance Gate applies to every subsequent release without exception.

Closing Instruction to the Engineering Leadership Team

You are now the authors of the Engineering Constitution and the Engineering Playbook. Author them as if the platform's survival depends on them, because it does. Author them as if every clause will be read by a reviewer at three o'clock in the morning during an incident, because it will be. Author them as if a build agent will implement against them without the benefit of your unspoken context, because it will. And author them as if the Chief Systems Architect will refuse to release any feature that cannot be traced to a clause within them, because that is exactly what will happen. The implementation loop is closed. The specification loop is now open. Begin.

Appendix A — RFC 2119 Vocabulary Reference

This Directive, and every document derived from it, uses the requirement vocabulary defined in IETF RFC 2119. The vocabulary is reproduced here for reference; the authoritative source is RFC 2119 itself (<https://datatracker.ietf.org/doc/html/rfc2119>). The Engineering Constitution and Engineering Playbook SHALL use this vocabulary with the precise meanings below, and SHALL NOT substitute synonyms (“will” for “shall”, “can” for “may”) that erode the precision of the requirement.

Term	Meaning
MUST	This word, or the terms “REQUIRED” or “SHALL”, mean that the definition is an absolute requirement of the specification.
MUST NOT	This phrase, or the phrase “SHALL NOT”, mean that the definition is an absolute prohibition of the specification.
SHOULD	This word, or the adjective “RECOMMENDED”, mean that there may exist valid reasons in particular circumstances to ignore a particular item, but the full implications must be understood and carefully weighed before choosing a different course.
SHOULD NOT	This phrase, or the phrase “NOT RECOMMENDED”, mean that there may exist valid reasons in particular circumstances when the particular behaviour is acceptable or even useful, but the full implications should be understood and the case carefully weighed before implementing any behaviour described with this label.
MAY	This word, or the adjective “OPTIONAL”, mean that an item is truly optional. One vendor may choose to include the item because a particular marketplace requires it or because the vendor feels that it enhances the product while another vendor may omit the same item.

In this Directive, “SHALL” is used for obligations on the Engineering Leadership Team (the audience of this Directive), and “MUST” is used for obligations that the resulting Engineering Constitution SHALL impose on implementers. The distinction is deliberate: this Directive governs the authors of the Constitution, and the Constitution governs the implementers of the platform. The vocabulary makes the chain of obligation explicit.

Appendix B — Glossary of Standards and Authorities

The following standards, protocols, and authoritative references are cited or implicated by this Directive. The Engineering Constitution SHALL cite the canonical authority for each when operationalizing the corresponding requirement, and SHALL pin the specific version where the authority is versioned. This glossary is normative for citation purposes.

ANSI/NISO Z39.96-2021 (JATS). Journal Article Tag Suite, the XML vocabulary for representing the full content and metadata of scholarly journal articles. Current version 1.3 (2021). Maintained by NISO; tag library hosted by NLM/NCBI. Canonical: <https://www.niso.org/standards-committees/jats> ; tag library: <https://jats.nlm.nih.gov/>

Crossref. The largest DOI registration agency for scholarly content. Provides the REST API, Deposit API, Schema Library, Content Negotiation, Similarity Check, Event Data, and webhooks. Canonical: <https://www.crossref.org/> ; documentation: <https://www.crossref.org/documentation/>

DOI (Digital Object Identifier). Persistent identifier scheme built on the Handle System. Each DOI is 10.<registrant>/<suffix> and resolves via the global Handle resolution service. Canonical: <https://www.doi.org/> ; RFC 7669 defines DOI structure and resolution.

ORCID. Open Researcher and Contributor Identifier; non-profit registry of persistent researcher identifiers. Public API and Member API (OAuth 2.0 three-legged). Current version ORCID API 3.0. Canonical: <https://orcid.org/>

CLOCKSS. Controlled LOCKSS; a dark archive jointly governed by publishers and libraries. Content ingested with no access unless a trigger event occurs. Canonical: <https://clockss.org/>

LOCKSS. Lots Of Copies Keep Stuff Safe; open-source peer-to-peer preservation network founded at Stanford (1998). Canonical: <https://www.lockss.org/>

Portico. Not-for-profit (Ithaka) dark-archive service for e-journals, e-books, and digitized collections; preserves via normalization. Canonical: <https://www.portico.org/>

OAI-PMH. Open Archives Initiative Protocol for Metadata Harvesting (v2.0). Defines six verbs: Identify, ListMetadataFormats, ListSets, ListIdentifiers, ListRecords, GetRecord. Canonical: <https://www.openarchives.org/pmh/>

SWORD v3. Repository deposit protocol (v3.0, September 2021). Defines Service Document, Deposit Receipt, and status/statement endpoints. Canonical: <https://sword.cottagelabs.com/>

COUNTER 5.1.1. Code of Practice for recording and exchanging online usage statistics for electronic resources. Defines TR, JR, PR, IR, DR reports. Canonical: <https://www.countermetrics.org/>

DOAJ. Directory of Open Access Journals; community-curated whitelist of fully open-access journals. Canonical: <https://doaj.org/>

OpenAIRE. European e-infrastructure aggregating research outputs linked to EU funding; co-developed the Principles of Open Scholarly Infrastructure (POSI). Canonical: <https://www.openaire.eu/>

PREMIS. Preservation Metadata Maintenance Activity; the de facto international standard for preservation metadata. Data Dictionary v3 current. Canonical: <https://www.loc.gov/standards/premis/>

COPE. Committee on Publication Ethics; publishes ethical guidelines for peer reviewers and editors. Canonical: <https://publicationethics.org/>

OJS (Open Journal Systems). Open-source journal management software from the Public Knowledge Project (PKP) at Simon Fraser University. The incumbent against which Opus Publica is positioned. Canonical: <https://pkp.sfu.ca/ojs/>

RFC 2119. Key words for use in RFCs to indicate requirement levels. Defines MUST/SHALL/SHOULD/MAY. Canonical: <https://datatracker.ietf.org/doc/html/rfc2119>

RFC 9110. HTTP Semantics (June 2022; obsoletes RFC 7232). Defines conditional requests, ETags, If-None-Match, If-Match. Canonical: <https://datatracker.ietf.org/doc/html/rfc9110>

NIST SP 800-92. Guide to Computer Security Log Management. The canonical guidance for audit log generation, transmission, storage, protection, retention, and review. Canonical: <https://csrc.nist.gov/pubs/sp/800/92/final>

12-Factor App. Methodology for building production-grade cloud applications (Heroku). Twelve factors covering codebase, dependencies, config, backing services, build/release/run, processes, port binding, concurrency, disposability, dev/prod parity, logs, admin processes. Canonical: <https://12factor.net/>

Google SRE. Site Reliability Engineering discipline formalizing SLIs, SLOs, and error budgets. Canonical: <https://sre.google/sre-book/>

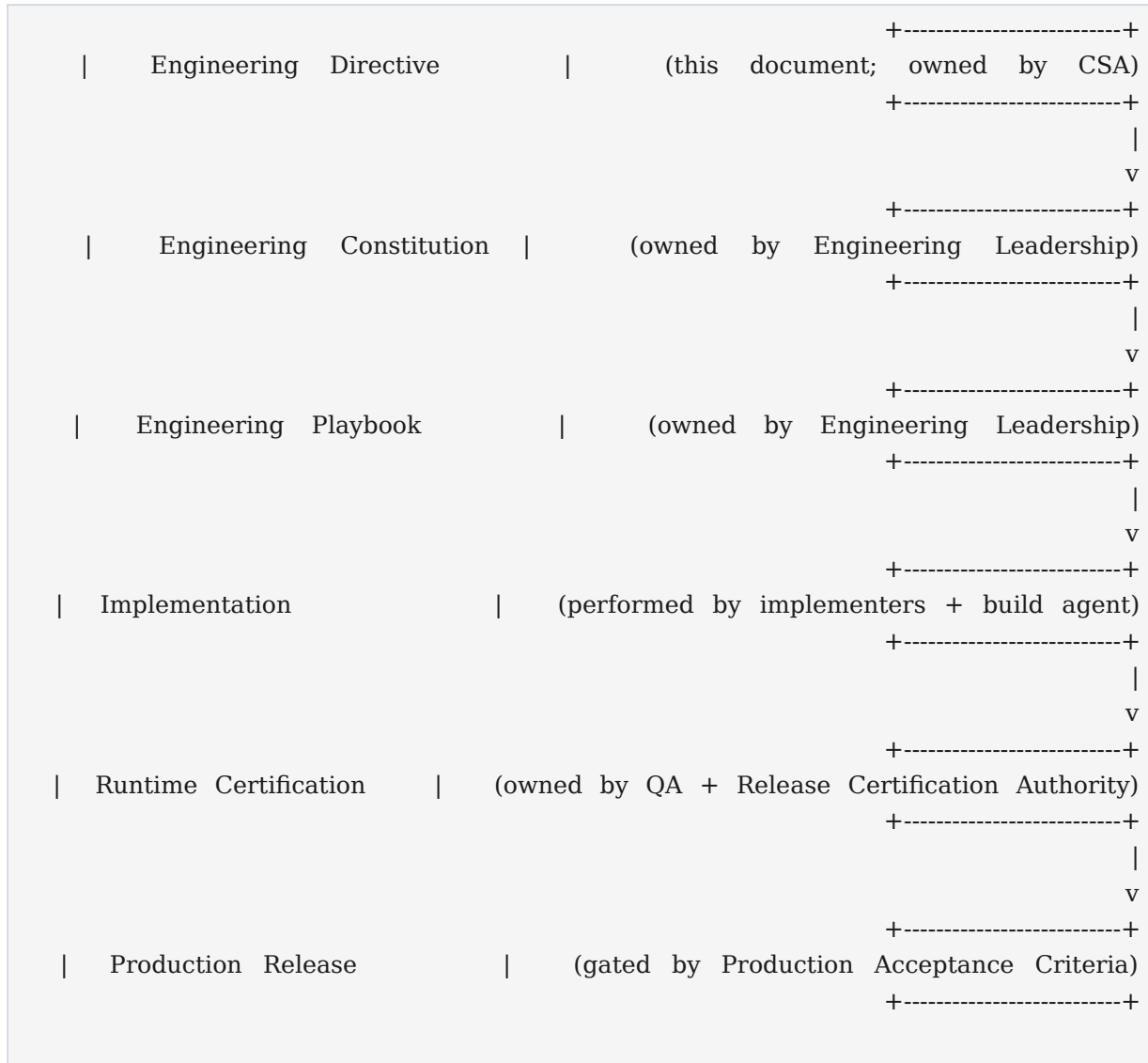
Conway's Law. Organizations which design systems are constrained to produce designs that are copies of their communication structures. The Inverse Conway Maneuver designs teams to produce the desired architecture. Canonical: <https://martinfowler.com/bliki/ConwaysLaw.html>

Runtime Verification. Discipline of observing production traces against a formal specification. Authoritative reference: Leucker & Schallhart, "A Brief Account of Runtime Verification," Journal of Logic and Algebraic Programming 78(5), 2009.

Evidence-Based Software Engineering (EBSE). Methodology formalizing evidence-based engineering decisions: systematic literature reviews, primary studies, meta-analysis. Authoritative reference: Kitchenham, Dyba, Jorgensen (2014).

Appendix C — The Engineering Flow

The Engineering Flow is the mandatory sequence of artifacts and gates through which every feature of Opus Publica SHALL pass. No step may be skipped; no step may be performed out of order. The flow is reproduced here as a single reference diagram and as a stage-by-stage table, so that it can be cited in code review, in certification, and in incident postmortems.



Stage	Owner	Entry Criteria	Exit Criteria	Gate Type
Engineering Directive	Chief Systems Architect	Need for a binding engineering contract.	Directive signed and published.	One-time issuance; amended by governance.
Engineering Constitution	Engineering Leadership	Directive in force; chapter owner assigned.	All 24 chapters authored, reviewed, signed off.	Specification gate.

Engineering Playbook	Engineering Leadership	Constitution chapter complete.	All 10 chapters authored, reviewed, exercised.	Operationalization gate.
Implementation	Implementer + build agent	Constitution clause and Playbook checklist identified.	Code complete, reviewed, CI passing, runtime smoke passing.	Implementation gate.
Runtime Certification	QA + Release Certification Authority	Implementation merged to release branch.	Five pillars verified; artifact set complete; RCA sign-off.	Certification gate (Production Acceptance Gate).
Production Release	Release Certification Authority	Certification complete; rollback tested.	Release deployed; post-deploy verification passing.	Release gate.

Appendix D — Certification Checklist Template

The following checklist SHALL be completed for every feature submitted for certification. The checklist is the operational form of Chapter 8 (Certification Standards). A feature is certifiable when every item is marked satisfied with a pointer to evidence, or marked not-applicable with a justification. Incomplete checklists block certification.

#	Checklist Item	Satisfied / N/A	Evidence Reference
1	Constitution clause implemented (cite by stable identifier).		
2	Playbook checklist item satisfied (cite by stable identifier).		
3	Runtime execution log captured in production-equivalent environment.		
4	Invariant monitor outputs show no violations during execution.		
5	Visual evidence (screenshots/recordings) of every user-visible state.		
6	Database verification: before/after persisted state; no unrelated changes.		
7	Storage verification: artifacts produced, content-addressed, retrievable.		
8	API verification: request/response recordings; contract tests pass.		
9	Idempotency tested: retries produce same end-state.		
10	Rollback tested: feature		

	can be rolled back without data loss.		
11	Audit log entries present for every state transition.		
12	No fallback hacks introduced (Rule 7.5).		
13	No magic strings introduced (Rule 7.4).		
14	No duplicated logic introduced (Rule 7.1).		
15	Owner declared in CODEOWNERS; on-call rotation declared.		
16	Telemetry exposes SLIs for the feature.		
17	Documentation updated and reviewed.		
18	Release Certification Authority sign-off recorded.		

The checklist SHALL be stored in the certification repository alongside the artifact set, and SHALL be referenced from the release record. A release whose checklist contains an unsatisfied item without a recorded justification is, by definition, an uncertified release and SHALL NOT be deployed.

Appendix E — Document Control and Revision History

E.1 Document Control

Field	Value
Document Identifier	OP-DIR-RC1-001
Document Title	Opus Publica RC1 Engineering Directive
Version	1.0
Issuing Authority	Office of the Chief Systems Architect, Opus Publica
Audience	Engineering Leadership Team
Classification	Mandatory — Engineering Leadership
Effective Date	09 August 2026
Supersedes	None (first issuance)
Next Review Date	09 November 2026 (quarterly)
Owner	Chief Systems Architect

E.2 Revision History

Version	Date	Author	Change
1.0	09 August 2026	Chief Systems Architect	Initial issuance. Establishes the Constitution-for-the-Constitution, the two Foundational Principles (Runtime Truth; Production Acceptance Gate), the ten chapters of the Directive, and the eight RC1 deliverables.

E.3 Acknowledgements

This Directive draws on the established literature of enterprise software engineering, scholarly publishing infrastructure, and site reliability engineering. The standards and authorities cited in Appendix B are the foundation on which the Directive's requirements rest; the Engineering Constitution SHALL cite them in turn when operationalizing the corresponding requirements. The discipline this Directive establishes is

not novel; it is the application of mature engineering practice to a platform that has, until now, been built without it.

The two Foundational Principles — the Principle of Runtime Truth and the Production Acceptance Gate — are the direct response to the specific failure mode this project has exhibited: an implementation loop in which specification lagged implementation, and in which runtime evidence was subordinated to static analysis. The principles are written to close that loop permanently, and to ensure that the platform's future is built on specification and evidence rather than on assumption and correction.

— *Issued under the authority of the Chief Systems Architect, Opus Publica* —